



**WYDZIAŁ FIZYKI
i INFORMATYKI STOSOWANEJ**
Uniwersytet Łódzki

Karol Mazur

Kierunek: informatyka

Specjalność: informatyka stosowana

Ścieżka dydaktyczna: sztuczna inteligencja

Numer albumu: 394248

Algorytmy kwantowe dla wybranych problemów grafowych

Praca magisterska

wykonana pod kierunkiem

dr hab. Kordiana A. Smolińskiego

w Katedrze Fizyki Teoretycznej

WFiIS UŁ

Łódź 2026

Spis treści

1	Wstęp	4
2	Podstawy teoretyczne	5
2.1	Problem najkrótszej ścieżki	5
2.2	Algorytm Dijkstry	6
2.3	Znajdowanie minimum	7
2.3.1	Wersja naiwna	7
2.3.2	Wersja z kopcem binarnym	8
2.3.3	Wersja kwantowa	9
2.4	Rodzaje grafów	15
2.4.1	Graf gęsty	16
2.4.2	Graf o średniej gęstości	17
2.4.3	Graf rzadki	18
2.4.4	Przypadek szczególny	19
2.5	Reprezentacja grafów	20
3	Opis implementacji	21
3.1	Tworzenie grafów	22
3.1.1	Generowanie grafu gęstego	22
3.1.2	Generowanie grafu o średniej gęstości	23
3.1.3	Generowanie grafu rzadkiego	23
3.1.4	Generowanie przypadku szczególnego	23
3.2	Implementacja algorytmu Dijkstry	24
3.2.1	Wersja naiwna	24
3.2.2	Wersja z kopcem binarnym	25
3.2.3	Wersja kwantowa	26

3.3	Analiza wyników	27
3.3.1	Ogólna analiza wyników	27
3.3.2	Analiza wyników wersji kwantowej	28
3.3.3	Tworzenie wykresów	29
4	Analiza wyników implementacji klasycznych	30
4.1	Wyniki wersji naiwnej	30
4.2	Wyniki wersji z kopcem binarnym	34
4.2.1	Graf rzadki	34
4.2.2	Graf o średniej gęstości i graf gęsty	34
4.2.3	Przypadek szczególny	35
4.2.4	Odchylenie standardowe wersji z kopcem	40
4.3	Bezpośrednie porównanie średnich wartości	42
4.4	Porównanie odchylenia standardowego	44
5	Analiza poprawności wersji kwantowej	46
5.1	Skuteczność wyszukiwania minimum z limitem	47
5.2	Skuteczność algorytmu Dijkstry z limitem	49
5.3	Skuteczność wyszukiwania minimum bez limitu	50
5.4	Skuteczność algorytmu Dijkstry bez limitu	51
5.5	Zależność pomiędzy błędnym minimum a wynikiem algorytmu Dijkstry . .	52
5.5.1	Dlaczego błędne minimum nie zawsze prowadzi do błędnego wyniku	53
6	Analiza wyników implementacji kwantowej	55
6.1	Wyniki dla wersji z limitem	56
6.1.1	Średnia liczba operacji	56
6.1.2	Odchylenie standardowe	57
6.2	Wyniki dla wersji bez limitu	58
6.2.1	Średnia liczba operacji	58
6.2.2	Odchylenie standardowe	59
6.3	Porównanie wersji z limitem i bez limitu	60
6.3.1	Porównanie średniej liczby operacji	60
6.3.2	Porównanie odchylenia standardowego	61
6.3.3	Dlaczego koszt wersji z limitem i bez limitu jest podobny	61

6.4	Zależność pomiędzy kosztem a poprawnością	63
7	Porównanie implementacji klasycznych i kwantowej	64
7.1	Porównanie średniej liczby operacji	64
7.1.1	Graf rzadki	65
7.1.2	Graf o średniej gęstości	65
7.1.3	Graf gęsty	65
7.1.4	Przypadek szczególny	65
7.2	Porównanie odchylenia standardowego	66
7.2.1	Graf rzadki	66
7.2.2	Graf o średniej gęstości	67
7.2.3	Graf gęsty	67
7.2.4	Przypadek szczególny	67
8	Podsumowanie	68
9	Opis metod i technologii wykorzystanych w pracy	69
9.1	Język programowania	69
9.2	Zewnętrzne paczki pythonowe	69
10	Instrukcja instalacji i obsługi	70
10.1	Instalacja	70
10.2	Instrukcja obsługi	70
10.2.1	Generowanie grafów	71
10.2.2	Analiza grafów	71
10.2.3	Analiza statystyczna wyników	71
10.2.4	Generowanie wykresów	71

Wstęp

Duża klasa problemów życia codziennego może być modelowanych przez odpowiadające im problemy grafowe. Wiele problemów grafowych jest NP-zupełnych lub NP-trudnych; są też problemy, dla których istnieją algorytmy wielomianowe. Dla wielu tych algorytmów kluczowym etapem jest przeprowadzenie wyszukiwania, np. minimalnej drogi. Ten etap można znacząco przyspieszyć wykorzystując kwantowy algorytm wyszukiwania minimum, będący uogólnieniem znanego kwantowego algorytmu wyszukiwania Grovera.

Celem tej pracy jest porównanie klasycznego i kwantowego wariantu algorytmu Dijkstry, służącego do znajdowania najkrótszej drogi w grafie. Algorytm ten zostanie zaimplementowany na trzy różne sposoby, a następnie zostanie sprawdzone, jak te podejścia radzą sobie na różnych rodzajach i rozmiarach grafów.

W pierwszej części pracy przedstawione zostaną podstawowe zagadnienia teoretyczne, obejmujące definicję grafów, ich rodzaje wykorzystywane w przeprowadzonych badaniach oraz zasadę działania algorytmu Dijkstry. Omówione zostaną również różne sposoby implementacji tego algorytmu. W dalszej części zaprezentowane zostaną wyniki przeprowadzonych eksperymentów wraz z ich analizą statystyczną, umożliwiającą ocenę rzeczywistej wydajności poszczególnych implementacji w zależności od charakterystyki analizowanych grafów. Na końcu pracy zamieszczono instrukcję instalacji i konfiguracji środowiska oraz wymaganych bibliotek, umożliwiającą samodzielne uruchomienie aplikacji i odtworzenie przedstawionych eksperymentów.

Podstawy teoretyczne

2.1 Problem najkrótszej ścieżki

Problem wyznaczania najkrótszej ścieżki należy do podstawowych zagadnień teorii grafów. Jego celem jest znalezienie ścieżki pomiędzy dwoma wierzchołkami grafu, której suma wag wszystkich krawędzi jest minimalna. Problem ten znajduje zastosowanie w wielu dziedzinach informatyki, takich jak systemy nawigacji, routing w sieciach komputerowych, logistyka, planowanie tras czy analiza sieci społecznościowych.

Formalnie graf definiowany jest jako para [1, s. 2]

$$G = (V, E) \tag{2.1}$$

gdzie (V) oznacza zbiór wierzchołków, natomiast (E) zbiór krawędzi łączących pary wierzchołków. W przypadku grafów ważonych każdej krawędzi przypisana jest waga określająca koszt przejścia pomiędzy dwoma sąsiadującymi wierzchołkami. Jeżeli wszystkie wagi są nieujemne, jednym z najbardziej efektywnych algorytmów rozwiązujących problem najkrótszych ścieżek z początkowego wierzchołka jest algorytm Dijkstry.

2.2 Algorytm Dijkstry

Algorytm Dijkstry [2] został po raz pierwszy przedstawiony w 1959 roku przez Edsger W. Dijkstra. Jest to algorytm zachłanny, który znajduje najkrótsze ścieżki od jednego wierzchołka źródłowego do wszystkich pozostałych wierzchołków w grafie o nieujemnych wagach krawędzi. Podstawową ideą algorytmu jest stopniowe powiększanie zbioru odwiedzonych wierzchołków, dla których wyznaczono już najkrótszą możliwą odległość od źródła. W każdej kolejnej iteracji wybierany jest wierzchołek o najmniejszej znanej odległości spośród nieodwiedzonych, po czym wykonywana jest operacja relaksacji wszystkich wychodzących z niego krawędzi.

Relaksacja polega na sprawdzeniu, czy przejście przez aktualnie analizowany wierzchołek prowadzi do krótszej ścieżki do sąsiada. Jeżeli zachodzi zależność [3, s. 649]

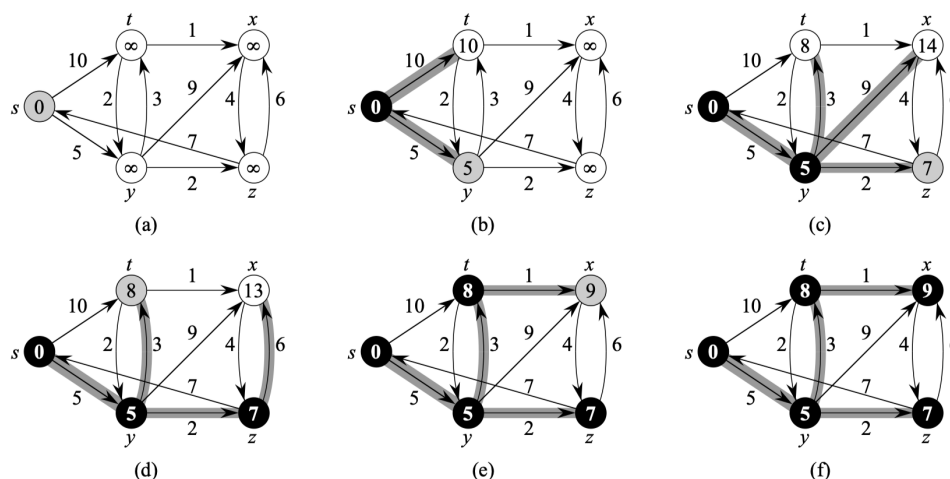
$$d(v) > d(u) + w(u, v) \quad (2.2)$$

wartość $d(v)$ zostaje zaktualizowana. Algorytm kończy działanie po odwiedzeniu wszystkich osiągalnych wierzchołków.

Algorytm 1 Algorytm Dijkstry

1. Zainicjalizuj odległości wszystkich wierzchołków od wierzchołka źródłowego.
2. Utwórz pusty zbiór odwiedzonych wierzchołków $S \leftarrow \emptyset$.
3. Umieść wszystkie wierzchołki grafu w zbiorze Q .
4. Dopóki zbiór Q nie jest pusty, wykonuj:
 - (a) Wybierz i usuń ze zbioru Q wierzchołek u o najmniejszej aktualnej odległości od źródła.
 - (b) Dodaj wierzchołek u do zbioru odwiedzonych S .
 - (c) Dla każdego sąsiada v wierzchołka u wykonaj relaksację krawędzi (u, v) .

Opracowanie własne na podstawie [3, s. 658]



Rysunek 2.1: Przykład działania algorytmu Dijkstry. [3, s. 659]

2.3 Znajdowanie minimum

Najbardziej kosztownym etapem algorytmu Dijkstry jest wyznaczenie kolejnego nieodwiedzonego wierzchołka o najmniejszej odległości. Sposób w jaki jest to zaimplementowane znacząco wpływa na złożoność całego algorytmu Dijkstry. Dlatego w celu porównania różnych podejść w niniejszej pracy wykorzystano trzy wersje algorytmu Dijkstry z różnymi sposobami znajdowania minimum:

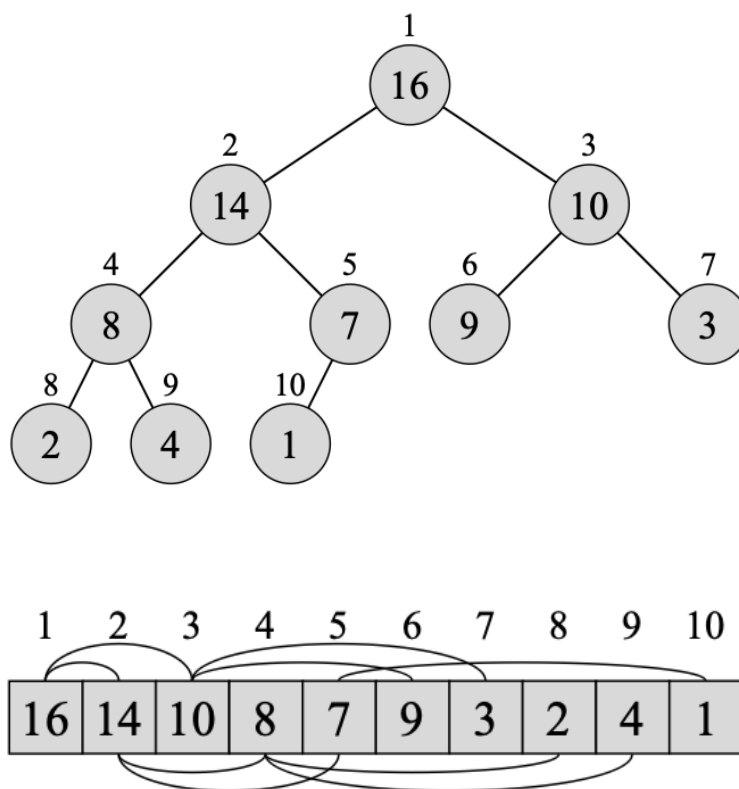
- wersję naiwną,
- wersję z wykorzystaniem kopca binarnego,
- wersję wykorzystującą kwantowy algorytm wyszukiwania minimum.

2.3.1 Wersja naiwna

W wersji naiwnej wybór kolejnego wierzchołka realizowany jest poprzez liniowe przeszukanie tablicy odległości w celu znalezienia najmniejszej wartości spośród wszystkich nieodwiedzonych wierzchołków. Operacja ta jest wykonywana w każdej iteracji algorytmu Dijkstry, a ponieważ liczba iteracji jest proporcjonalna do liczby wierzchołków V , całkowity koszt wyszukiwania minimum wynosi $O(V^2)$. Pomimo niskiej wydajności, wersja naiwna cechuje się prostotą implementacji i często wykorzystywana jest jako punkt odniesienia podczas porównywania bardziej zaawansowanych wariantów algorytmu jak na przykład kopiec binarny.

2.3.2 Wersja z kopcem binarnym

W celu przyspieszenia operacji wyboru kolejnego wierzchołka można zastosować kopiec binarny przechowujący pary składające się z identyfikatora wierzchołka oraz aktualnej wartości odległości. Kopiec binarny jest strukturą danych reprezentowaną w postaci niemal pełnego drzewa binarnego, w którym każdy węzeł posiada co najwyżej dwoje dzieci. W kopcu minimalnym spełniona jest własność, zgodnie z którą wartość klucza każdego węzła jest nie większa od wartości kluczy jego potomków. Oznacza to, że element o najmniejszym kluczu znajduje się zawsze w korzeniu drzewa. Jego pobranie oraz usunięcie wymaga czasu $O(\log V)$. Wstawienie nowego elementu lub aktualizacja wartości klucza również wykonywane są w czasie $O(\log V)$ poprzez odpowiednie odtworzenie własności kopca. Dzięki zastosowaniu kopca binarnego wyszukiwanie wierzchołka o najmniejszej odległości nie wymaga liniowego przeszukiwania wszystkich elementów [3, r. 6].



Rysunek 2.2: Przykład kopca binarnego *max* wraz z listą jego wartości i kluczy [3, s. 151]. W niniejszej pracy do implementacji Dijkstry wykorzystywana jest wersja *min* kopca binarnego

2.3.3 Wersja kwantowa

W implementacji kwantowej klasyczna operacja wyboru nieodwiedzonego wierzchołka o najmniejszej wartości odległości została zastąpiona kwantowym algorytmem wyszukiwania minimum stworzonym przez Dürra i Høyera [4]. Algorytm ten stanowi rozwinięcie kwantowego algorytmu wyszukiwania Grovera [5], wykorzystując uogólniony algorytm wyszukiwania opracowany przez Boyera, Brassarda, Høyera i Tappa (BBHT) [6]. W przeciwieństwie do klasycznego algorytmu Grovera, który zakłada znajomość liczby rozwiązań, algorytm ten umożliwia skuteczne wyszukiwanie również w przypadku nieznanej liczby rozwiązań.

Algorytm Dürra-Høyera realizuje wyszukiwanie minimum poprzez iteracyjne wywołanie algorytmu BBHT w celu znalezienia elementu o wartości mniejszej od aktualnego kandydata na minimum. Po znalezieniu takiego elementu zostaje on nowym kandydatem, a proces jest powtarzany aż do momentu, gdy z dużym prawdopodobieństwem nie istnieje już element o mniejszej wartości.

Algorytm Grovera

Algorytm Grovera to kwantowa procedura sprawdzania obecności klucza w bazie danych. W nim baza danych nie ma ustalonej struktury jak w klasycznych bazach gdzie dane są zorganizowane np. w struktury drzewiaste. Jego celem jest znalezienie elementu/elementów z wykorzystaniem wyroczni (*oracle*), która potrafi rozpoznać poprawne rozwiązania. W przeciwieństwie do klasycznego przeszukiwania liniowego wymagającego średnio $O(N)$ sprawdzeń, algorytm Grovera osiąga złożoność $O(\sqrt{N})$ [5].

Ważnym ograniczeniem podstawowej wersji algorytmu jest konieczność wykonania odpowiedniej liczby iteracji. Liczba ta zależy od liczby elementów spełniających zadany warunek określony przez wyrocznię i w ogólnym przypadku nie jest znana. Zbyt mała liczba iteracji nie zapewnia wystarczającego wzmocnienia amplitudy rozwiązania, natomiast zbyt duża powoduje jej ponowne zmniejszenie, co obniża prawdopodobieństwo poprawnego wyniku. Problem ten został rozwiązany w algorytmie Boyera, Brassarda, Høyera i Tappa (BBHT), który zostanie omówiony w kolejnym podrozdziale.

Algorytm 2 Algorytm Grovera

1. Przygotuj rejestr kwantowy w stanie równomiernej superpozycji wszystkich N możliwych stanów.
2. Powtarzaj operator Grovera odpowiednią liczbę razy:
 - (a) Zastosuj wyrocznie (*oracle*), która zmienia znak amplitudy stanów odpowiadających rozwiązaniom problemu.
 - (b) Zastosuj operator dyfuzji (*diffusion transform*), który odbija amplitudy wszystkich stanów względem ich średniej wartości, zwiększając amplitudę stanów oznaczonych przez wyrocznie i zmniejszając amplitudy pozostałych.
3. Wykonaj pomiar rejestru kwantowego.
4. Zwróć zmierzony stan jako wynik wyszukiwania.

Opracowanie własne na podstawie [5]

W pierwszym kroku tworzona jest superpozycja wszystkich możliwych stanów. Żeby to uzyskać w rejestrze kwantowym złożonym z n kubitów stosujemy n -krotnie bramkę Hadamarda. [7, s. 162, wz. (4.66)]

$$|\psi\rangle = H^{\otimes n}|00\cdots 0\rangle = (H|0\rangle)^{\otimes n} = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right)^{\otimes n} = \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |i\rangle. \quad (2.3)$$

Dla stanu kubitów złożonego z $n=3$ kubitów odpowiada to temu zapisowi [7, s. 278, wz. (5.366)]

$$|\psi'\rangle = \frac{1}{2\sqrt{2}}(|000\rangle + |001\rangle + |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle). \quad (2.4)$$

Wszystkie amplitudy prawdopodobieństwa mają wartość $1/\sqrt{N}$ co oznacza że każdy stan jest jednakowo prawdopodobny w przypadku pomiaru.

Kolejny etap stanowi iteracyjne wykonywanie operatora Grovera. Każda iteracja składa się z dwóch operacji. Pierwszą z nich jest działanie wyroczni, która identyfikuje stany spełniające zadany warunek poprzez odwrócenie znaku ich amplitudy. Jest to funkcja która odpowiada za wskazanie szukanego elementu [7, s. 162, wz. (4.67)].

$$f(i) = \begin{cases} 1, & \text{jeśli } i \text{ reprezentuje szukany element,} \\ 0, & \text{w przeciwnym przypadku.} \end{cases} \quad (2.5)$$

Oczekuje się, że działanie tej funkcji implementowanej przez unitarny operator U_f można opisać wzorem [7, s. 162, wz. (4.68)]

$$U_f(|i\rangle|j\rangle) = |i\rangle|j \oplus f(i)\rangle. \quad (2.6)$$

Przypadek w którym funkcja ta zmienia znak na rejestrze kwantowym przy trzeciej amplitudzie można zapisać w ten sposób [7, s. 278, wz. (5.367)]

$$|\psi''\rangle = \frac{1}{2\sqrt{2}}(|000\rangle + |001\rangle - |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle). \quad (2.7)$$

Wykonanie pomiaru bezpośrednio po zastosowaniu wyroczni nie pozwala jeszcze na wskazanie poprawnego rozwiązania. Wynika to z faktu, że wyrocznia zmienia na ujemny znak amplitudy stanu odpowiadającego szukanemu elementowi. Podczas pomiaru wyznaczane jest natomiast prawdopodobieństwo otrzymania danego stanu, które jest równe kwadratowi modułu jego amplitudy. W rezultacie informacja o znaku zostaje utracona. Z tego powodu konieczne jest wykonanie kolejnego etapu algorytmu operatora dyfuzji nazywanego również operatorem obrotu wokół średniej. Jego zadaniem jest zwiększenie amplitud stanów oznaczonych przez wyrocznię kosztem amplitud pozostałych stanów.

Definicja operatora obrotu wokół średniej, oznaczonego jako A przybiera postać [7, s. 163, wz. (4.71)]

$$A = 2|\psi\rangle\langle\psi| - I. \quad (2.8)$$

gdzie $|\psi\rangle$ to stan opisany wzorem (2.3)

Po zastosowaniu operatora U_f otrzymuje się stan [7, s. 163, wz. (4.72)]

$$|\psi_1\rangle = |\psi\rangle - \frac{2}{\sqrt{2^n}}|i_{U_f}\rangle. \quad (2.9)$$

Odpowiada to takiej bramce kwantowej przedstawionej w formie macierzy [7, s. 277, wz. (5.364)]

$$G_{L \times L} = \begin{pmatrix} \frac{2}{L} - 1 & \frac{2}{L} & \cdots & \frac{2}{L} \\ \frac{2}{L} & \frac{2}{L} - 1 & \cdots & \frac{2}{L} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{2}{L} & \frac{2}{L} & \cdots & \frac{2}{L} - 1 \end{pmatrix}, \quad L = 2^n. \quad (2.10)$$

Całość działania operatora obrotu wokół średniej G na stanie (2.7) można przedstawić w ten sposób [7, s. 278, wz. (5.368)]

$$|\psi'''\rangle = G|\psi''\rangle = \begin{pmatrix} -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} \end{pmatrix} \begin{pmatrix} \frac{1}{2\sqrt{2}} \\ \frac{1}{2\sqrt{2}} \\ -\frac{1}{2\sqrt{2}} \\ \frac{1}{2\sqrt{2}} \\ \frac{1}{2\sqrt{2}} \\ \frac{1}{2\sqrt{2}} \\ \frac{1}{2\sqrt{2}} \\ \frac{1}{2\sqrt{2}} \end{pmatrix} = \begin{pmatrix} \frac{1}{4\sqrt{2}} \\ \frac{1}{4\sqrt{2}} \\ \frac{5}{4\sqrt{2}} \\ \frac{1}{4\sqrt{2}} \\ \frac{1}{4\sqrt{2}} \\ \frac{1}{4\sqrt{2}} \\ \frac{1}{4\sqrt{2}} \\ \frac{1}{4\sqrt{2}} \end{pmatrix}. \quad (2.11)$$

Powtarzanie tych dwóch operacji (wyroczeni i dyfuzji) prowadzi do stopniowego zwiększania prawdopodobieństwa uzyskania poprawnego rozwiązania. Po wykonaniu odpowiedniej liczby iteracji przeprowadzany jest pomiar rejestru kwantowego, podczas którego prawdopodobieństwo uzyskania danego stanu jest równe kwadratowi modułu jego amplitudy. Dzięki wcześniejszemu wzmocnieniu amplitudy stanu odpowiadającego rozwiązaniu zostaje on uzyskany z wysokim prawdopodobieństwem.

Algorytm BBHT

Algorytm opracowany przez Boyera, Brassarda, Høyer i Tappa stanowi rozszerzenie algorytmu Grovera przeznaczone dla sytuacji, w której liczba rozwiązań problemu nie jest znana. W przeciwieństwie do klasycznej wersji algorytmu nie można z góry określić optymalnej liczby iteracji operatora Grovera. Zamiast tego algorytm stopniowo zwiększa zakres możliwej liczby iteracji.[6]

Algorytm 3 Algorytm BBHT

1. Zainicjalizuj parametry algorytmu: ustaw $m = 1$ oraz wybierz współczynnik $\lambda = \frac{6}{5}$.
2. Wylosuj liczbę całkowitą j równomiernie z przedziału $\{0, 1, \dots, m - 1\}$.
3. Przygotuj rejestr kwantowy w stanie superpozycji wszystkich N możliwych stanów.
4. Wykonaj j iteracji algorytmu Grovera (Alg. 2).
5. Wykonaj pomiar rejestru kwantowego i otrzymaj stan i .
6. Jeżeli $T[i] = x$, zakończ działanie algorytmu i zwróć znalezione rozwiązanie.
7. W przeciwnym razie zwiększ wartość parametru m zgodnie z zależnością

$$m = \min(\lambda m, \sqrt{N}),$$

a następnie wróć do kroku 2.

Opracowanie własne na podstawie [6]

Losowy dobór liczby iteracji pozwala uniknąć sytuacji, w której zastosowanie zbyt dużej jej ilości zmniejszyłoby prawdopodobieństwo uzyskania poprawnego rozwiązania. Dzięki stopniowemu zwiększaniu parametru m algorytm zachowuje kwadratowe przyspieszenie względem algorytmów klasycznych, osiągając oczekiwany czas działania rzędu

$$O\left(\sqrt{\frac{N}{t}}\right) \tag{2.12}$$

gdzie t oznacza liczbę rozwiązań problemu [6].

Kwantowy algorytm wyszukiwania minimum

Algorytm wyszukiwania minimum, zaproponowany przez Dürra i Høyer, wykorzystuje algorytm BBHT jako podprocedurę służącą do znajdowania elementów o wartości mniejszej od aktualnie przyjętego progu. Algorytm ten iteracyjnie aktualizuje indeks elementu o najmniejszej dotychczas znalezionej wartości. Dzięki zastosowaniu kwantowego przeszukiwania możliwe jest znalezienie minimum tablicy w oczekiwanym czasie rzędu $\mathcal{O}(\sqrt{N})$ przy prawdopodobieństwie sukcesu wynoszącym co najmniej $\frac{1}{2}$ [4].

Algorytm 4 Kwantowy algorytm wyszukiwania minimum

1. Wylosuj indeks y z przedziału $\{0, 1, \dots, N - 1\}$ i przyjmij go jako początkowy próg.
2. Powtarzaj poniższe kroki do momentu przekroczenia limitu czasu działania opisanego w (2.13):

- (a) Przygotuj rejestr kwantowy w stanie równomiernej superpozycji wszystkich stanów.
- (b) Oznacz wszystkie stany odpowiadające indeksom j , dla których zachodzi

$$T[j] < T[y].$$

- (c) Uruchom algorytm BBHT (Alg. 3) w celu znalezienia jednego z oznaczonych elementów.
- (d) Wykonaj pomiar rejestru kwantowego i otrzymaj indeks y' .
- (e) Jeżeli

$$T[y'] < T[y],$$

to zaktualizuj próg, przyjmując $y \leftarrow y'$.

3. Po zakończeniu iteracji zwróć indeks y jako wynik działania algorytmu.

Opracowanie własne na podstawie [4]

Łączny oczekiwany czas działania algorytmu do momentu, w którym zmienna y będzie wskazywać indeks elementu minimalnego, wynosi[4]

$$m_0 = \frac{45}{4}\sqrt{N} + \frac{7}{10}\lg^2 N. \quad (2.13)$$

2.4 Rodzaje grafów

Wpływ rodzaju grafu na wydajność algorytmu jest jednym z najważniejszych aspektów niniejszej pracy. Analiza zostanie przeprowadzona na czterech różnych rodzajach grafów: grafie rzadkim, średnio gęstym, gęstym oraz specjalnie przygotowanym grafie, który wymusza dużą liczbę relaksacji. We wszystkich analizowanych przypadkach rozpatrywane są grafy nieskierowane, bez pętli własnych, o nieujemnych wagach krawędzi. W przypadku tego ostatniego rodzaju grafu wagi krawędzi będą ściśle określone. Natomiast dla pozostałych grafów wagi będą losowane z przedziału 1 do n^2 , gdzie n oznacza liczbę wierzchołków.

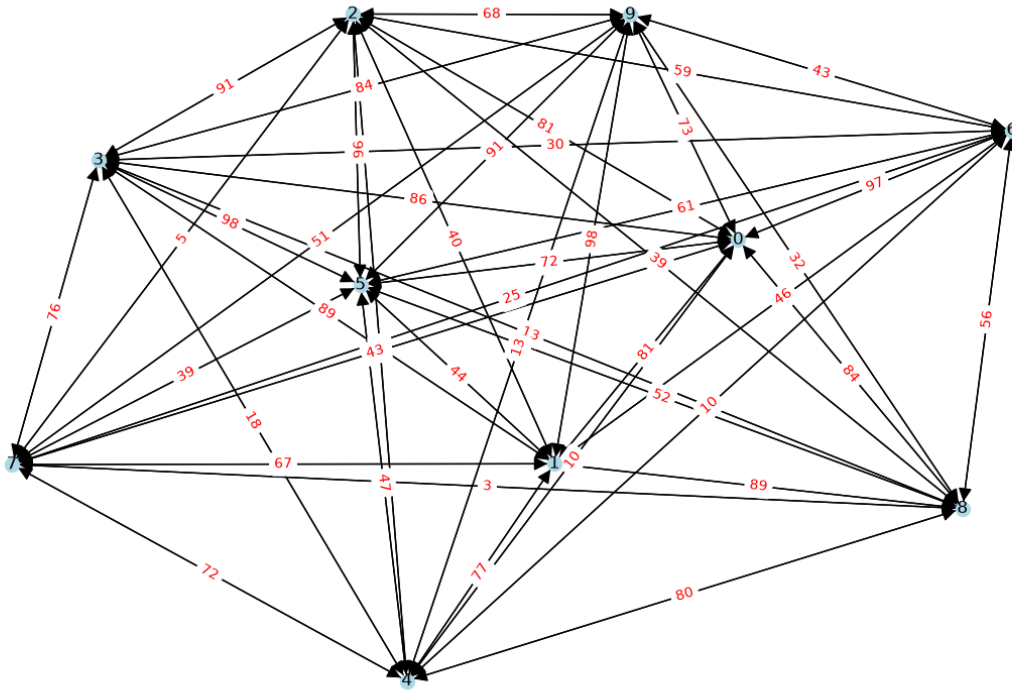
2.4.1 Graf gęsty

Graf gęsty zawiera liczbę krawędzi bliską maksymalnej [8]

$$|E| = O(|V|^2). \quad (2.14)$$

W niniejszej pracy graf gęsty jest grafem pełnym nieskierowanym, w którym liczba krawędzi jest równa maksymalnej liczbie krawędzi dopuszczalnej dla grafu nieskierowanego bez pętli [9]

$$E = \frac{V(V-1)}{2}. \quad (2.15)$$



Rysunek 2.3: Przykład grafu "dense" analizowanego w pracy

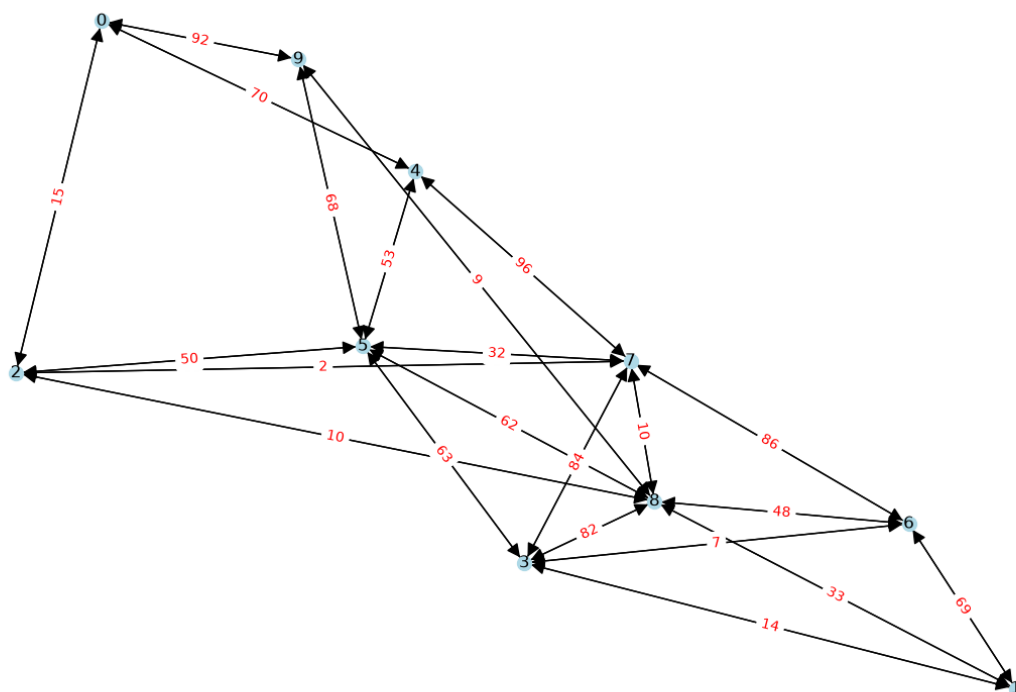
2.4.2 Graf o średniej gęstości

Graf średnio gęsty zajmuje pośrednią pozycję pomiędzy grafami rzadkimi i pełnymi. Liczba krawędzi jest znacznie większa niż liczba wierzchołków, jednak nadal nie osiąga wartości maksymalnej jak w grafach gęstych [8]

$$|E| = O(|V|^2). \quad (2.16)$$

W niniejszej pracy jako graf o średniej gęstości przyjęto graf nieskierowany, którego liczba krawędzi stanowi połowę maksymalnej liczby krawędzi

$$E = \frac{V(V-1)}{4}. \quad (2.17)$$



Rysunek 2.4: Przykład grafu "half-edges" analizowanego w pracy

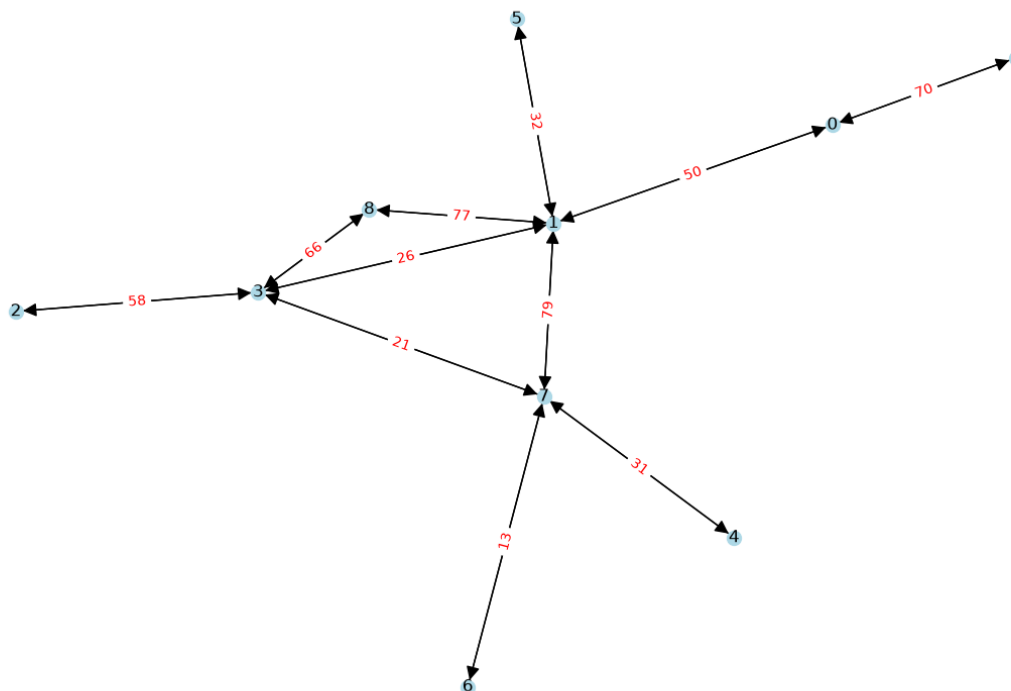
2.4.3 Graf rzadki

Graf rzadki charakteryzuje się niewielką liczbą krawędzi względem maksymalnej liczby możliwych połączeń [8]

$$|E| = O(|V|). \quad (2.18)$$

Przykładami grafów rzadkich są sieci drogowe, drzewa oraz wiele rzeczywistych sieci komputerowych. W niniejszej pracy jako graf rzadki przyjęto graf nieskierowany, w którym liczba krawędzi jest równa liczbie wierzchołków

$$E = V. \quad (2.19)$$



Rysunek 2.5: Przykład grafu "sparse" analizowanego w pracy

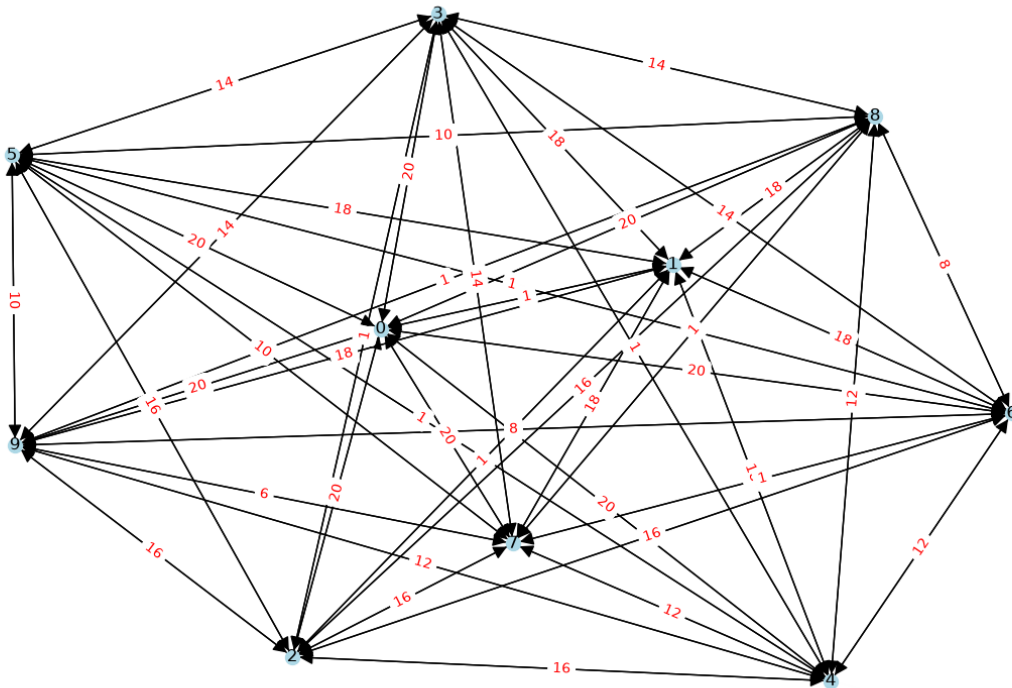
2.4.4 Przypadek szczególny

Przypadek szczególny jest specjalnie przygotowanym grafem pełnym. W odróżnieniu od pozostałych analizowanych grafów jego struktura oraz wagi krawędzi nie są losowe, lecz zostały dobrane w taki sposób, aby wymusić dużą liczbę operacji relaksacji wykonywanych przez algorytm Dijkstry. Tak jak graf gęsty, przypadek szczególny zawiera maksymalną liczbę krawędzi dla grafu nieskierowanego bez pętli [9]

$$E = \frac{V(V-1)}{2}. \quad (2.20)$$

Wagi krawędzi są wyznaczone zgodnie z następującą zależnością

$$w(i, j) = \begin{cases} 1, & \text{gdy } j = i + 1, \\ 2(|V| - i), & \text{gdy } j > i + 1. \end{cases} \quad (2.21)$$



Rysunek 2.6: Przykład grafu "special case" analizowanego w pracy

2.5 Reprezentacja grafów

Graf może być reprezentowany na różne sposoby, przy czym do najczęściej stosowanych metod należą macierz sąsiedztwa oraz lista sąsiedztwa. W celu zilustrowania obu sposobów reprezentacji wykorzystano graf przedstawiony na rysunku 2.4.

$$A = \begin{pmatrix} 0 & 0 & 15 & 0 & 70 & 0 & 0 & 0 & 0 & 92 \\ 0 & 0 & 0 & 14 & 0 & 0 & 69 & 0 & 33 & 0 \\ 15 & 0 & 0 & 0 & 0 & 50 & 0 & 2 & 10 & 0 \\ 0 & 14 & 0 & 0 & 0 & 63 & 7 & 84 & 82 & 0 \\ 70 & 0 & 0 & 0 & 0 & 53 & 0 & 96 & 0 & 0 \\ 0 & 0 & 50 & 63 & 53 & 0 & 0 & 32 & 62 & 68 \\ 0 & 69 & 0 & 7 & 0 & 0 & 0 & 86 & 48 & 0 \\ 0 & 0 & 2 & 84 & 96 & 32 & 86 & 0 & 10 & 0 \\ 0 & 33 & 10 & 82 & 0 & 62 & 48 & 10 & 0 & 9 \\ 92 & 0 & 0 & 0 & 0 & 68 & 0 & 0 & 9 & 0 \end{pmatrix} \quad (2.22)$$

Macierz sąsiedztwa przedstawia graf w postaci dwuwymiarowej tablicy, w której element znajdujący się na przecięciu i -tego wiersza i j -tej kolumny określa wagę krawędzi pomiędzy odpowiadającymi im wierzchołkami.

$$\begin{aligned} 0 &: \{(2, 15), (9, 92), (4, 70)\}, \\ 1 &: \{(6, 69), (8, 33), (3, 14)\}, \\ 2 &: \{(8, 10), (0, 15), (7, 2), (5, 50)\}, \\ 3 &: \{(5, 63), (7, 84), (8, 82), (1, 14), (6, 7)\}, \\ 4 &: \{(7, 96), (5, 53), (0, 70)\}, \\ 5 &: \{(7, 32), (8, 62), (3, 63), (9, 68), (4, 53), (2, 50)\}, \\ 6 &: \{(8, 48), (1, 69), (7, 86), (3, 7)\}, \\ 7 &: \{(4, 96), (6, 86), (5, 32), (2, 2), (8, 10), (3, 84)\}, \\ 8 &: \{(2, 10), (6, 48), (1, 33), (9, 9), (5, 62), (7, 10), (3, 82)\}, \\ 9 &: \{(8, 9), (5, 68), (0, 92)\}. \end{aligned} \quad (2.23)$$

W implementacji przedstawionej w niniejszej pracy graf został zapisany w postaci listy sąsiedztwa. W niej dla każdego wierzchołka przechowywana jest lista wszystkich wierzchołków z nim połączonych wraz z odpowiadającymi im wagami krawędzi.

Opis implementacji

W niniejszym rozdziale przedstawiono sposób implementacji rozwiązania opisanego od strony teoretycznej w poprzednim rozdziale. Implementacja obejmuje generowanie grafów, trzy warianty algorytmu Dijkstry, analizę otrzymanych wyników oraz tworzenie wykresów umożliwiających ich porównanie.

Poszczególne etapy działania programu zostały rozdzielone, żeby mogły być uruchamiane niezależnie. Dane pomiędzy nimi przekazywane są przy użyciu plików JSON. Wygenerowane grafy są zapisywane do plików, a następnie odczytywane przez moduły wykonujące odpowiednie wersje algorytmu Dijkstry. Wyniki zapisywane są w plikach JSON, które następnie wykorzystywane są przez moduły do analizy statystycznej. Jej wyniki stanowią z kolei dane wejściowe dla modułów odpowiedzialnych za generowanie wykresów. Cały proces przetwarzania danych w implementacji można przedstawić w następujący sposób:

generowanie grafów → zapis do JSON → wykonanie algorytmów → zapis wyników do JSON → analiza statystyczna → zapis statystyk do JSON → generowanie wykresów.

Taki podział na niezależne moduły ułatwia przeprowadzanie eksperymentów z wykorzystaniem poszczególnych wersji algorytmu Dijkstry, analizę uzyskanych wyników oraz ponowne generowanie wykresów bez konieczności wykonywania całego procesu od początku.

3.1 Tworzenie grafów

Pierwszym etapem jest przygotowanie grafów stanowiących dane wejściowe dla wszystkich analizowanych wariantów algorytmu. W części teoretycznej przedstawiono cztery rodzaje grafów wykorzystywane w pracy: graf rzadki, graf o średniej gęstości, graf gęsty oraz przypadek szczególny. Ich właściwości, a także przyjęte liczby krawędzi, opisano w rozdziale dotyczącym rodzajów grafów 2.4.

Liczba wierzchołków grafów oraz liczba różnych grafów generowanych dla każdego badanego rozmiaru są określane za pomocą pliku konfiguracyjnego. Pozwala to na łatwe dostosowanie parametrów bez konieczności modyfikowania kodu źródłowego.

Grafy przechowywane są w postaci listy sąsiedztwa, zgodnie z reprezentacją opisaną wcześniej w rozdziale 2.5. Po wygenerowaniu każdy graf jest zapisywany do pliku JSON. Jego nazwa zawiera informacje pozwalające określić typ grafu, jego rozmiar oraz numer. Dzięki temu wszystkie implementacje algorytmu Dijkstry mogą korzystać z tych samych danych wejściowych.

3.1.1 Generowanie grafu gęstego

Graf gęsty został zaimplementowany jako graf pełny. Oznacza to, że dla każdej pary różnych wierzchołków tworzona jest dokładnie jedna krawędź. Liczba krawędzi jest zatem zgodna z zależnością

$$|E| = \frac{|V|(|V| - 1)}{2}, \quad (3.1)$$

przedstawioną również w części teoretycznej w rozdziale 2.4.1. Dla każdej utworzonej krawędzi losowana jest jej waga. Przykład grafu gęstego przedstawiono na rys. 2.3.

3.1.2 Generowanie grafu o średniej gęstości

W przypadku grafu o średniej gęstości liczba tworzonych krawędzi stanowi połowę maksymalnej liczby krawędzi grafu pełnego, zgodnie z zależnością przedstawioną wcześniej w rozdziale 2.4.2

$$|E| = \frac{|V|(|V| - 1)}{2}. \quad (3.2)$$

Kolejne krawędzie tworzone są pomiędzy losowo wybranymi parami różnych wierzchołków. Podczas generowania sprawdzane jest, czy dana krawędź nie została utworzona wcześniej, dzięki czemu graf nie zawiera wielokrotnych krawędzi pomiędzy tą samą parą wierzchołków. Wagi krawędzi losowane są z zakresu określonego w części teoretycznej. Przykład wykorzystywanego grafu o średniej gęstości przedstawiono na rys. 2.4.

3.1.3 Generowanie grafu rzadkiego

Graf rzadki generowany jest zgodnie z definicją przyjętą w części teoretycznej w rozdziale 2.4.3, zgodnie z którą liczba krawędzi jest równa liczbie wierzchołków

$$|E| = |V|. \quad (3.3)$$

Mechanizm losowania krawędzi oraz ich wag jest analogiczny jak w przypadku grafu o średniej gęstości. Różnica polega na większej liczbie tworzonych połączeń. Przykład grafu tego typu przedstawiono na rys. 2.5.

3.1.4 Generowanie przypadku szczególnego

Ostatnim generowanym typem jest przypadek szczególny przedstawiony na rys. 2.6. Podobnie jak graf gęsty jest on grafem pełnym, jednak wagi jego krawędzi nie są losowane. Sposób ich wyznaczania odpowiada zależności przedstawionej w części teoretycznej w rozdziale 2.4.4.

$$w(i, j) = \begin{cases} 1, & \text{gdy } j = i + 1, \\ 2(|V| - i), & \text{gdy } j > i + 1. \end{cases}$$

3.2 Implementacja algorytmu Dijkstry

Podstawowy sposób działania algorytmu Dijkstry został przedstawiony w alg. 1. Wszystkie trzy implementacje użyte w części działają według tego samego schematu. Różnią się przede wszystkim sposobem wykonania operacji wyboru nieodwiedzonego wierzchołka o najmniejszej aktualnie znanej odległości.

W celu porównania różnych metod wyszukiwania minimum zaimplementowano trzy warianty:

- wersję naiwną,
- wersję wykorzystującą kopiec binarny,
- wersję wykorzystującą kwantowy algorytm wyszukiwania minimum.

3.2.1 Wersja naiwna

W wersji naiwnej wyszukiwanie minimum realizowane poprzez liniowe przeglądanie wszystkich wierzchołków grafu. W każdej iteracji algorytmu Dijkstry wybierany jest spośród nieprzetworzonych wierzchołków ten, którego aktualnie znana odległość od wierzchołka źródłowego jest najmniejsza. Sposób ten odpowiada wersji naiwnej opisanej w części teoretycznej w rozdziale 2.3.1.

Na potrzeby zliczania operacji podczas wyszukiwania wprowadzono zmienną przechowującą liczbę wykonanych operacji. Za pojedynczą operację przyjęto sprawdzenie jednego wierzchołka w wewnętrznej pętli wyszukiwania. Licznik ten jest zwiększany o 1 dla każdego sprawdzanego wierzchołka, niezależnie od tego, czy zostanie on nowym kandydatem na minimum. Ponieważ podczas pojedynczego wyszukiwania przeglądane jest zawsze n wierzchołków, liczba zliczonych operacji wynosi

$$C_{\text{search}} = n. \quad (3.4)$$

Wyszukiwanie wykonywane jest n razy, ponieważ zewnętrzna pętla algorytmu również wykonuje n iteracji. Całkowita liczba operacji zliczonych dla jednego uruchomienia wersji naiwnej wynosi więc

$$C_{\text{naive}} = n^2. \quad (3.5)$$

Po zakończeniu działania algorytmu wartość zmiennej liczącej liczbę operacji jest zapisywana, a następnie przekazywana w pliku JSON do dalszej analizy statystycznej.

3.2.2 Wersja z kopcem binarnym

Druga klasyczna implementacja wykorzystuje kopiec binarny opisany w części teoretycznej w rozdziale 2.3.1. Przykładową strukturę kopca przedstawiono na rys. 2.2. W przeciwieństwie do wersji naiwnej wybór wierzchołka o najmniejszej odległości nie wymaga liniowego przeglądania wszystkich nieodwiedzonych wierzchołków.

Na potrzeby algorytmu zaimplementowano własną strukturę kopca, która zlicza wykonywane podczas jego działania operacje. Uwzględniane są ich dwa rodzaje:

- porównania wartości priorytetów,
- zamiany elementów wewnątrz kopca.

Operacje te są zliczane podczas wstawiania elementów do kopca oraz pobierania elementu o najmniejszym priorytecie.

W implementacji zastosowano podejście *lazy heap*. Początkowo do kopca wstawiany jest wyłącznie wierzchołek startowy z odległością równą zero. Pozostałe wierzchołki są dodawane dopiero w momencie znalezienia do nich pierwszej skończonej odległości. W przypadku znalezienia krótszej drogi podczas kolejnej relaksacji wartość przechowywana wcześniej w kopcu nie jest bezpośrednio modyfikowana za pomocą operacji *decrease-key*. Zamiast tego aktualizowana jest odległość danego wierzchołka, a do kopca dodawany jest nowy element zawierający tę odległość.

Wstawienie nowego elementu może wymagać ponownego uporządkowania kopca, co wiąże się z kolejnymi porównaniami i zamianami elementów. Takie same operacje mogą wystąpić podczas pobierania elementu o najmniejszym priorytecie. Dlatego liczba operacji dla wersji z kopcem jest sumą liczników obu operacji. Po zakończeniu algorytmu suma ta jest zapisywana do pliku JSON.

3.2.3 Wersja kwantowa

Trzecia implementacja algorytmu Dijkstry wykorzystuje kwantowy algorytm wyszukiwania minimum Dürra-Høyer'a przedstawiony w alg. 4. Algorytm ten wykorzystuje algorytm BBHT (alg. 3), który z kolei wykonuje określoną liczbę iteracji algorytmu Grovera (alg. 2).

Ogólna struktura algorytmu Dijkstry pozostaje taka sama jak w alg. 1. W każdej iteracji tworzona jest lista wierzchołków wraz z odpowiadającymi im odległościami. Lista może zawierać również wierzchołki o odległości równej nieskończoności. Takie wierzchołki mogą zostać pominięte podczas dostosowywania rozmiaru przestrzeni wyszukiwania dlatego liczba elementów jest następnie zwiększana lub zmniejszana do najbliższej odpowiedniej potęgi liczby 2. Wynika to ze sposobu reprezentacji stanów przez rejestr kwantowy. Rejestr składający się z n kubitów pozwala reprezentować 2^n stanów bazowych. Liczba operacji w wersji kwantowej wyznaczana jest na podstawie dwóch elementów:

- kosztu przygotowania rejestru kwantowego w stanie równomiernej superpozycji,
- liczby wykonanych iteracji algorytmu Grovera.

Dla przestrzeni wyszukiwania zawierającej N elementów potrzebny jest rejestr składający się z

$$n = \log_2 N \quad (3.6)$$

kubitów, ponieważ $N = 2^n$. Przygotowanie superpozycji stanów wymaga zastosowania bramki Hadamarda do każdego z n kubitów, zgodnie ze wzorem (2.3). Koszt przygotowania rejestru wynosi zatem

$$C_{\text{init}} = \log_2 N. \quad (3.7)$$

W procedurze BBHT losowana jest dodatkowo wartość j , określająca liczbę iteracji algorytmu Grovera wykonywanych w danej próbie. Koszt pojedynczej próby BBHT określono więc jako

$$C_{\text{BBHT}} = \log_2 N + j. \quad (3.8)$$

Pierwszy składnik odpowiada koszcie przygotowania superpozycji, natomiast drugi liczbie wykonanych iteracji algorytmu Grovera. Jeżeli algorytm BBHT wymaga kilku prób, ich koszty są sumowane. Algorytm Dürra-Høyer'a może następnie wielokrotnie wywoływać algorytm BBHT podczas znajdowania kolejnych elementów mniejszych od aktualnego

kandydata na minimum. Koszty tych wywołań są również sumowane. Ostatecznie liczba operacji zarejestrowana podczas działania algorytmu Dijkstry odpowiada sumie kosztów wszystkich prób BBHT wykonanych podczas wszystkich wywołań kwantowego wyszukiwania minimum.

Dodatkowo rejestrowane są: liczba wywołań algorytmu wyszukiwania minimum, liczba przypadków, w których minimum znalezione przez algorytm kwantowy różniło się od rzeczywistego minimum, oraz informacja o niepoprawności końcowego wyniku algorytmu Dijkstry. Po zakończeniu działania wszystkie zliczone wartości zapisywane są do pliku w formacie JSON.

3.3 Analiza wyników

Na tym etapie działania programu dane są wczytywane z plików JSON zapisanymi w poprzednim etapie opisanym w 3.2, a następnie poddawane analizie. Takie rozwiązanie pozwala na modyfikowanie sposobu obliczania statystyk bez konieczności ponownego uruchamiania wcześniejszych etapów programu.

3.3.1 Ogólna analiza wyników

Dla każdej grupy pomiarów, obejmującej naiwną implementację klasycznego algorytmu Dijkstry, implementację wykorzystującą kopiec binarny oraz kwantowy algorytm Dijkstry, obliczane są następujące statystyki dotyczące liczby operacji wykonywanych przez poszczególne algorytmy:

- średnia,
- mediana,
- odchylenie standardowe,
- wartość minimalna i maksymalna,

W dalszej analizie wyników szczególnie istotne są mediana oraz odchylenie standardowe. Mediana jest wykorzystywana, ponieważ jest mniej wrażliwa na pojedyncze wartości odstające niż średnia. Odchylenie standardowe pozwala natomiast określić sam rozrzut wyników. Ma to znaczenie przede wszystkim w przypadku wersji kwantowej, ale jest obliczane

też dla obu implementacji klasycznych, dzięki czemu możliwe jest porównanie stabilności wszystkich analizowanych wersji algorytmu.

Obliczone statystyki zapisywane są do kolejnych plików JSON, które stanowią dane wejściowe dla modułów odpowiedzialnych za tworzenie wykresów.

3.3.2 Analiza wyników wersji kwantowej

W przypadku wersji kwantowej przeprowadzana jest dodatkowa analiza związana z probabilistycznym działaniem algorytmu wyszukiwania minimum Dürra-Høyerera przedstawionego w algorytmie 4. Algorytm ten wykorzystuje algorytm BBHT (alg. 3), która wykonuje określoną liczbę iteracji algorytmu Grovera (alg. 2).

Oprócz analizy liczby wykonanych operacji opisanej w poprzednim podrozdziale 3.3.1, analizowane są również: liczba wywołań algorytmu wyszukiwania minimum, liczba przypadków, w których znalezione minimum różniło się od poprawnego minimum, oraz liczba niepoprawnych wyników samego algorytmu Dijkstry. Dla tych wartości obliczane są podstawowe statystyki, takie jak średnia, mediana, odchylenie standardowe, minimum oraz maksimum. Porównywana jest również liczba błędnie znalezionych minimów z liczbą niepoprawnych wyników algorytmu Dijkstry. Pozwala to sprawdzić, czy błędne znalezienie minimum zawsze prowadzi do niepoprawnego wyniku algorytmu Dijkstry.

Dodatkowo analizowana jest skuteczność algorytmu Dijkstry, określając liczbę poprawnych wyników wśród wszystkich otrzymanych wyników. W podobny sposób oceniana jest skuteczność algorytmu wyszukiwania minimum Dürra-Høyerera na podstawie tego, jak często znalezione minimum było zgodne z poprawnym minimum.

Oddzielnie analizowane są konfiguracje z limitem wynikającym ze wzoru (2.13) oraz bez limitu. Pozwala to sprawdzić, jaki wpływ zastosowanie limitu ma na liczbę wykonywanych operacji oraz na prawdopodobieństwo otrzymania poprawnego wyniku.

Wszystkie wyniki przeprowadzonej analizy są następnie zapisywane do plików JSON.

3.3.3 Tworzenie wykresów

Ostatnim etapem programu jest przygotowanie wykresów przedstawiających uzyskane wyniki. Dane potrzebne do ich utworzenia są odczytywane z plików JSON, które powstają podczas wcześniejszej analizy wyników opisanej w rozdziale 3.3. Rozwiązanie to pozwala zmieniać sposób prezentacji danych bez konieczności ponownego wykonywania wcześniejszych etapów programu.

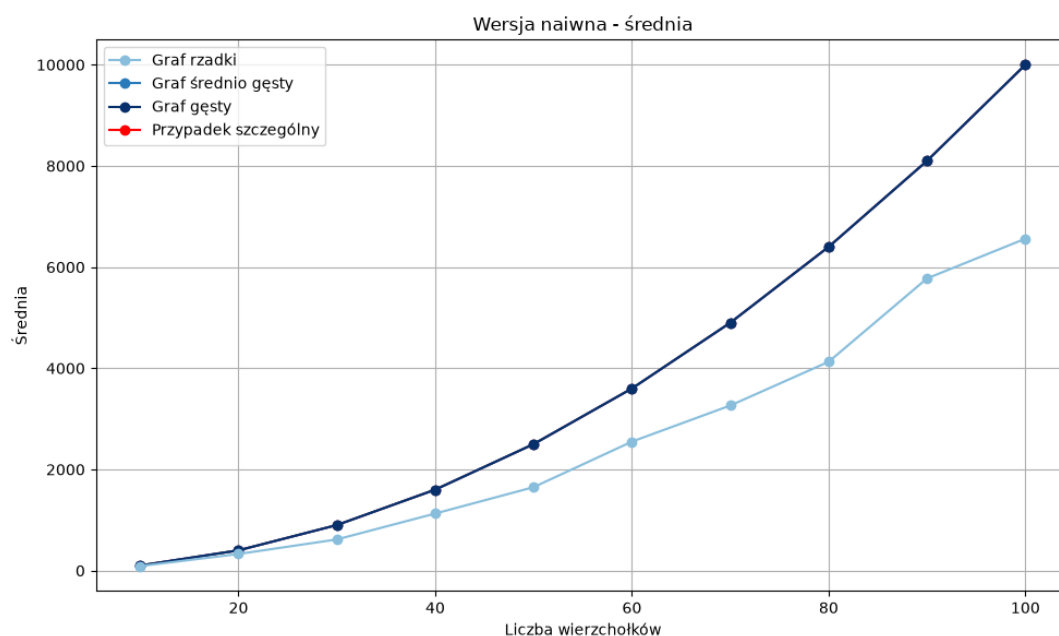
Dla wszystkich implementacji tworzone są wykresy przedstawiające średnią, medianę oraz odchylenie standardowe w zależności od liczby wierzchołków grafu. Pozwala to obserwować, jak wraz ze wzrostem rozmiaru grafu zmienia się liczba operacji.

Dla wersji kwantowej tworzone są również dodatkowe wykresy. Przedstawiają one liczbę błędnie znalezionych minimów, liczbę niepoprawnych wyników algorytmu Dijkstry, liczbę wywołań algorytmu wyszukiwania minimum Dürra-Høyera oraz skuteczność jego działania.

Analiza wyników implementacji klasycznych

W niniejszym rozdziale przeprowadzono analizę wyników otrzymanych dla dwóch klasycznych implementacji algorytmu Dijkstry: wersji naiwnej oraz wersji wykorzystującej kopiec binarny. Analizie poddano cztery rodzaje grafów przedstawione wcześniej w rozdziale 2.4: graf rzadki, graf o średniej gęstości, graf gęsty oraz przypadek szczególny. Porównywaną wielkością jest liczba operacji wyszukiwania minimum algorytmu Dijkstry.

4.1 Wyniki wersji naiwnej



Rysunek 4.1: Średnia liczba operacji dla naiwnej wersji algorytmu Dijkstry.

Na rys. 4.1 przedstawiono średnią liczbę operacji wykonywanych przez naiwną wersję algorytmu. Dla grafu o średniej gęstości, grafu gęstego oraz przypadku szczególnego widoczny jest ten sam wykres. Wynika to ze sposobu zliczania operacji w tej implementacji.

Podczas pojedynczej iteracji algorytmu sprawdzane jest wszystkie n wierzchołków, niezależnie od liczby krawędzi oraz ich wag. Jeżeli graf jest spójny, algorytm wykonuje n takich iteracji. Liczba zliczonych operacji jest zatem równa

$$C_{\text{naive}} = n \cdot n = n^2. \quad (4.1)$$

Dla przykładu, dla $n = 100$ otrzymywana jest wartość

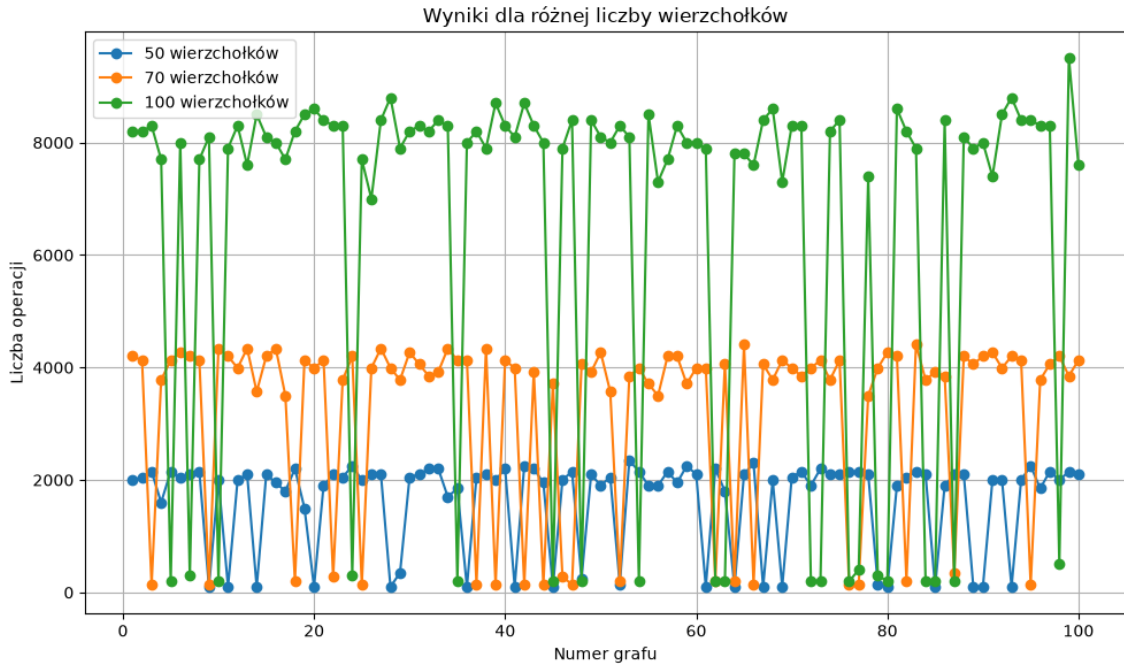
$$C_{\text{naive}} = 100^2 = 10000, \quad (4.2)$$

co odpowiada wartości widocznej na wykresie dla grafu średnio gęstego, gęstego oraz przypadku szczególnego.

Wyjątkiem jest graf rzadki. Jego średnia liczba operacji jest wyraźnie mniejsza od liczby operacji dla pozostałych typów grafów. Nie jest to spowodowane tym, że dla niego pojedyncza operacja wyszukiwania minimum wymagana mniejszej liczby operacji. Taki graf zawiera tylko

$$|E| = |V| \quad (4.3)$$

krawędzi i podczas generowania nie jest wymuszane, żeby graf był spójny. Może więc powstać sytuacja, w której z wierzchołka początkowego nie da się osiągnąć wszystkich pozostałych wierzchołków.



Rysunek 4.2: Liczba operacji wersji naiwnej dla kolejnych losowo wygenerowanych grafów rzadkich.

Na wykresie można zauważyć wyraźny podział wyników na dwie grupy. Jednocześnie występują pojedyncze przypadki, w których liczba operacji jest bardzo mała. Takie zachowanie wynika ze struktury losowych grafów rzadkich. W wielu przypadkach wierzchołek startowy i kolejne wierzchołki są połączone z wieloma innymi. Wtedy algorytm może dotrzeć do większości wierzchołków i wykonuje dużą liczbę iteracji. Jeżeli natomiast z wierzchołka startowego lub przez jego sąsiadów nie można przejść do wielu kolejnych wierzchołków, algorytm kończy działanie znacznie wcześniej bo nie dociera do wielu wierzchołków.

Można to zauważyć na podstawie liczby operacji. Dla $n = 100$ wartości liczby operacji gdy algorytm dociera do dużej liczby wierzchołków znajdują się w okolicach 8000. Wynika wtedy, że stosunek liczby operacji do wierzchołków wynosi

$$k \approx \frac{8000}{100} = 80. \quad (4.4)$$

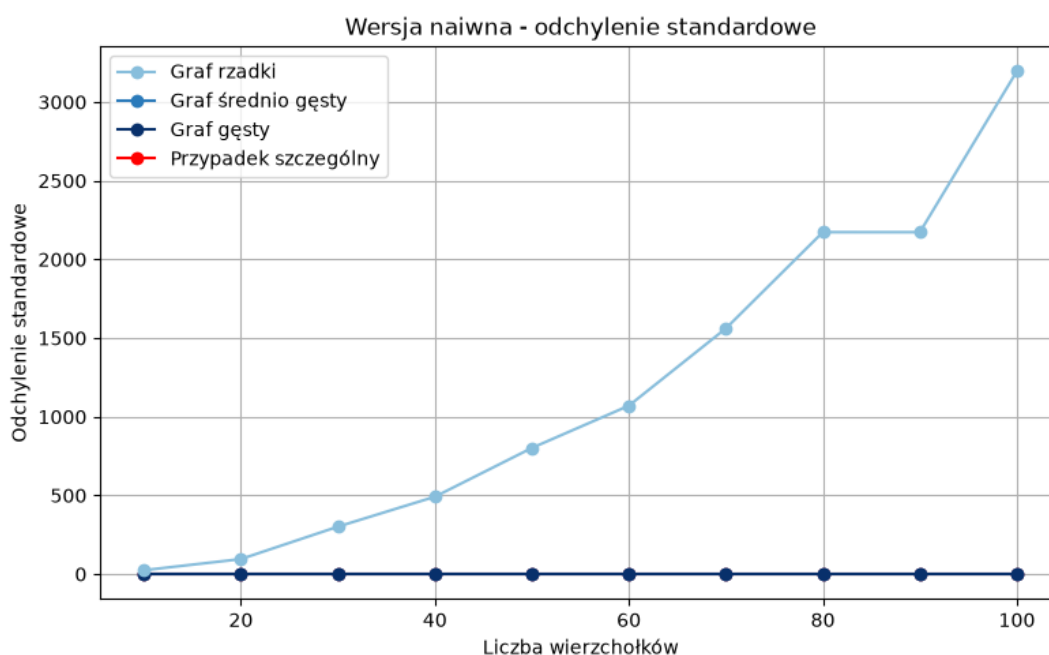
Oznacza to, że w tych przypadkach z wierzchołka początkowego osiągalnych jest około 80 spośród 100 wierzchołków. Analogicznie dla $n = 50$, przy wartości około 2000, otrzymuje się

$$k \approx \frac{2000}{50} = 40, \quad (4.5)$$

natomiast dla $n = 70$, przy około 4000 operacji,

$$k \approx \frac{4000}{70} \approx 57. \quad (4.6)$$

W każdym z tych przypadków osiągalnych jest około 80% wszystkich wierzchołków. Na wykresie są też przypadki, dla których koszt jest wielokrotnie mniejszy. Odpowiadają one grafom, w których algorytm kończy działanie znacznie wcześniej.

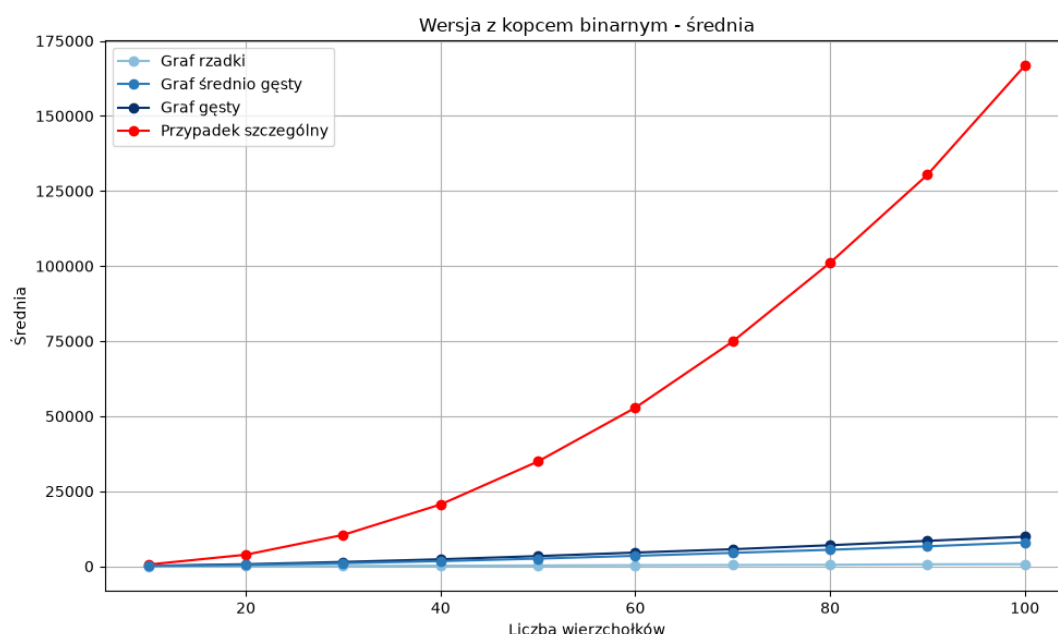


Rysunek 4.3: Odchylenie standardowe liczby operacji dla naiwnej wersji algorytmu Dijkstry.

Rysunek 4.3 to potwierdza. Dla grafu średnio gęstego, gęstego oraz przypadku szczególnego odchylenie standardowe wynosi zero. Każde uruchomienie dla ustalonej liczby wierzchołków wykonuje dokładnie taką samą liczbę operacji równą n^2 . Losowość wag i połączeń nie ma wpływu na koszt wyszukiwania w tablicy w celu znalezienia minimum.

Inaczej zachowują się grafy rzadkie. W ich przypadku widoczny jest bardzo duży wzrost odchylenia standardowego wraz ze wzrostem liczby wierzchołków. Powodem jest występowanie dwóch odległych grup wyników - tej gdzie przez algorytm jest osiągana duża liczba wierzchołków oraz drugiej gdzie algorytm kończy działanie bardzo wcześnie.

4.2 Wyniki wersji z kopcem binarnym



Rysunek 4.4: Średnia liczba operacji dla wersji algorytmu Dijkstry wykorzystującej kopiec binarny.

Wyniki przedstawione na rys. 4.4 bardzo różnią się od rezultatów wersji naiwnej. Liczba operacji zależy w tym przypadku nie tylko od liczby wierzchołków, ale też od struktury grafu, wag krawędzi oraz liczby udanych relaksacji.

4.2.1 Graf rzadki

Najmniejszy średni koszt wersji z kopcem uzyskano dla grafów rzadkich, ponieważ każdy wierzchołek posiada niewielką liczbę sąsiadów. Liczba możliwości znalezienia nowej, krótszej drogi do tego samego wierzchołka jest ograniczona.

4.2.2 Graf o średniej gęstości i graf gęsty

Wraz ze wzrostem gęstości grafu zwiększa się liczba krawędzi analizowanych podczas relaksacji. Zwiększa się również liczba sytuacji, w których dla danego wierzchołka zostaje znaleziona odległość mniejsza od wartości wyznaczonej wcześniej. Z tego powodu dla grafu średnio gęstego i gęstego liczba operacji wykonywanych wewnątrz kopca jest znacznie większa niż w przypadku grafu rzadkiego.

4.2.3 Przypadek szczególny

Najbardziej wyróżniającym się wynikiem jest bardzo duża liczba operacji wykonywana przez wersję z kopcem dla przypadku szczególnego. Dla $n = 100$ średnia osiąga około $1.66 \cdot 10^5$ operacji, podczas gdy wersja naiwna wykonuje 10^4 operacji. Oznacza to ponad szesnastokrotną różnicę na korzyść wersji naiwnej.

W tym przypadku bardzo duże znaczenie ma sposób doboru wag krawędzi. Dla przypadku szczególnego zastosowano zależność opisaną we wzorze (4.7)

$$w(i, j) = \begin{cases} 1, & \text{gdy } j = i + 1, \\ 2(|V| - i), & \text{gdy } j > i + 1. \end{cases} \quad (4.7)$$

Powoduje ona, że najkrótsza droga od wierzchołka 0 prowadzi kolejno przez wierzchołki

$$0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow n - 1,$$

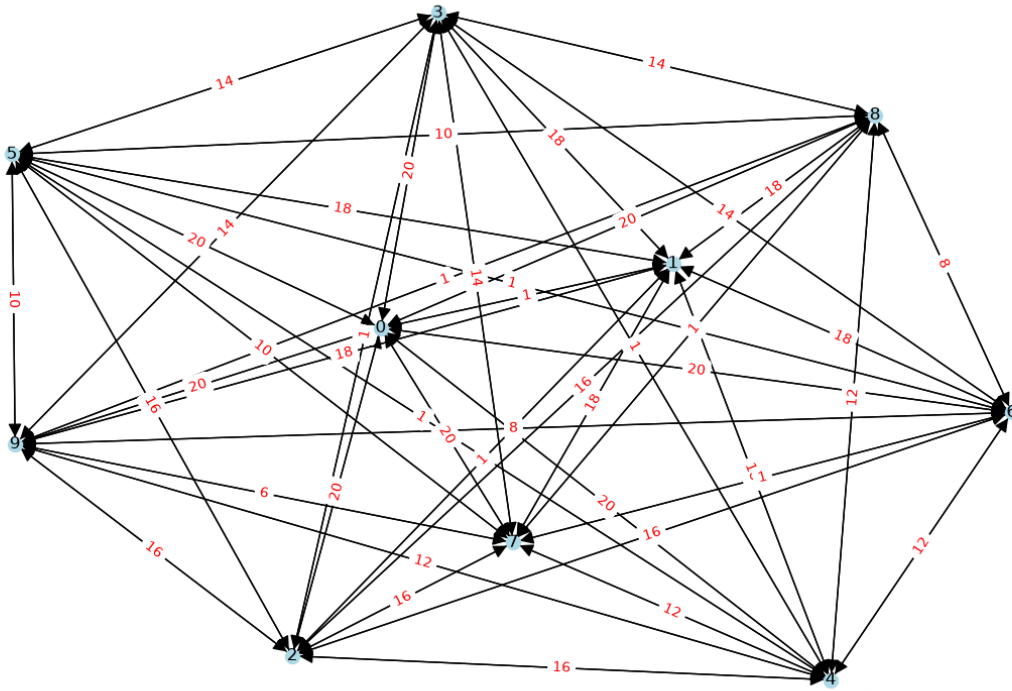
a odległości przyjmują wartości

$$d(0) = 0, \quad d(1) = 1, \quad d(2) = 2, \quad \dots, \quad d(n - 1) = n - 1.$$

Jednocześnie bezpośrednie krawędzie do dalszych wierzchołków mają duże wagi. Dzięki temu początkowo otrzymywane są duże wartości odległości, które następnie są wielokrotnie zmniejszane na etapie relaksacji.

Przykład działania dla dziesięciu wierzchołków

W celu dokładniejszego pokazania powodu dużej liczby operacji wykonywanych przez wersję wykorzystującą kopiec binarny można przeanalizować przypadek szczególny dla $n = 10$. Przykład takiego grafu przedstawiono na rys. 4.5.



Rysunek 4.5: Przykład przypadku szczególnego dla 10 wierzchołków.

Wagi krawędzi w tym grafie są określone zgodnie z zależnością

$$w(i, j) = \begin{cases} 1, & \text{gdy } j = i + 1, \\ 2(10 - i), & \text{gdy } j > i + 1. \end{cases}$$

Oznacza to, że krawędzie łączące kolejne wierzchołki

$$(0, 1), (1, 2), (2, 3), \dots, (8, 9)$$

mają wagę równą 1. Pozostałe krawędzie wychodzące od wierzchołka o mniejszym numerze mają odpowiednio większe wagi. Przykładowo dla wierzchołka 0 wszystkie krawędzie prowadzące dalej niż do wierzchołka 1 mają wagę

$$2(10 - 0) = 20.$$

Dla wierzchołka 1 odpowiednia wartość wynosi

$$2(10 - 1) = 18,$$

a dla wierzchołka 2

$$2(10 - 2) = 16,$$

Taką zależność jest też widoczna na rys. 4.5, gdzie występują między innymi wagi 20, 18, 16, 14, 12, 10, 8 oraz 6.

Gdy algorytm rozpoczyna działanie w wierzchołku 0, jego początkowa odległość wynosi

$$d(0) = 0.$$

Po przetworzeniu wierzchołka 0 krawędź prowadząca do wierzchołka 1 ma wagę 1, dlatego

$$d(1) = 1.$$

A do wszystkich pozostałych wierzchołków prowadzą krawędzie o wadze 20. Otrzymywane są więc odległości

$$d(2) = d(3) = \dots = d(9) = 20.$$

Po pierwszej relaksacji w kopcu znajdują się zatem między innymi wpisy

$$(1, 1), (20, 2), (20, 3), \dots, (20, 9).$$

Najmniejszą wartość jest dla $(1, 1)$, dlatego jako następny sprawdzany jest wierzchołek 1.

Krawędź $(1, 2)$ ma wagę 1, zatem odległość do wierzchołka 2 zostaje poprawiona z 20 do

$$d(2) = d(1) + 1 = 2.$$

Dla wszystkich dalszych wierzchołków $j > 2$ waga krawędzi wychodzącej z wierzchołka 1 wynosi 18. Nowy kandydat na odległość ma więc wartość

$$d(1) + 18 = 1 + 18 = 19.$$

Otrzymuje się zatem

$$d(3) = d(4) = \dots = d(9) = 19.$$

Wszystkie te wartości są mniejsze od wcześniejszej wartości 20. Do kopca dodawane są więc nowe wpisy

$$(2, 2), (19, 3), (19, 4), \dots, (19, 9).$$

W tym miejscu ujawnia się istotna cecha implementacji *lazy heap*. Poprzednie elementy

$$(20, 2), (20, 3), \dots, (20, 9)$$

nie są usuwane z kopca. Pozostają w nim jako wpisy nieaktualne.

Następnie z kopca pobierany jest wierzchołek 2, którego poprawna odległość wynosi

$$d(2) = 2.$$

Krawędź $(2, 3)$ ma wagę 1, dlatego

$$d(3) = 3.$$

Dla wierzchołków $4, \dots, 9$ waga odpowiednich krawędzi wychodzących z wierzchołka 2 wynosi

$$2(10 - 2) = 16.$$

Nowa odległość wynosi zatem

$$d(2) + 16 = 2 + 16 = 18.$$

Wcześniej wartości te były równe 19, dlatego ponownie następuje udana relaksacja:

$$d(4) = d(5) = \dots = d(9) = 18.$$

Do kopca trafia więc kolejny zestaw elementów

$$(3, 3), (18, 4), (18, 5), \dots, (18, 9).$$

Jednocześnie znajdują się w nim nadal wcześniejsze wpisy z odległościami 19 oraz 20.

Po przetworzeniu wierzchołka 3 sytuacja znowu się powtarza. Otrzymywana jest odległość

$$d(4) = 4,$$

a dla wierzchołków $5, \dots, 9$

$$d(j) = d(3) + 2(10 - 3) = 3 + 14 = 17.$$

Poprzednia wartość wynosiła 18, dlatego wszystkie te odległości ponownie zostają poprawione.

Pierwsze etapy działania algorytmu można więc przedstawić w ten sposób:

przetworzony wierzchołek	odległość do następnego wierzchołka	odległość do dalszych wierzchołków
0	$d(1) = 1$	20
1	$d(2) = 2$	19
2	$d(3) = 3$	18
3	$d(4) = 4$	17
4	$d(5) = 5$	16
5	$d(6) = 6$	15
6	$d(7) = 7$	14
7	$d(8) = 8$	13
8	$d(9) = 9$	12

Widoczny jest więc mechanizm działania algorytmu na przygotowanym grafie. Przetworzenie kolejnego wierzchołka powoduje poprawienie odległości do prawie wszystkich wierzchołków znajdujących się dalej w kolejności. Wartość kandydata zmniejsza się przy tym o 1 w stosunku do wartości uzyskanej w poprzedniej iteracji.

Dla przykładu odległość do wierzchołka 9 jest aktualizowana wielokrotnie:

$$20 \rightarrow 19 \rightarrow 18 \rightarrow 17 \rightarrow 16 \rightarrow 15 \rightarrow 14 \rightarrow 13 \rightarrow 9.$$

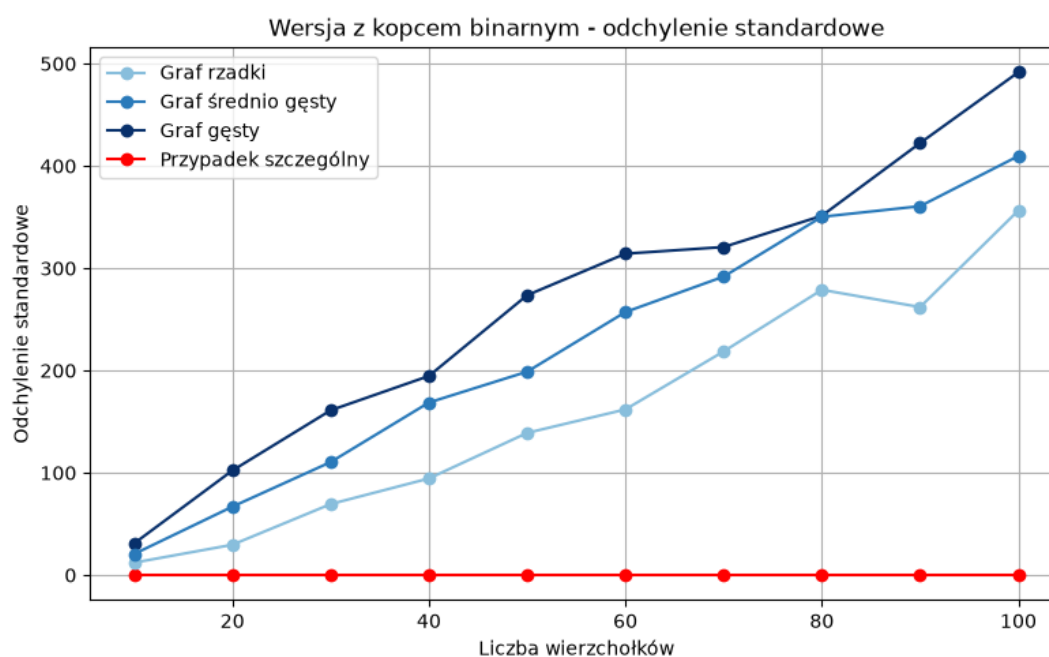
To samo jest dla pozostałych dalszych wierzchołków. Dlatego ten sam wierzchołek posiada w kopcu wiele wpisów odpowiadających kolejnym, coraz mniejszym odległościom.

Przykładowo dla wierzchołka 9 w trakcie działania algorytmu mogą pojawić się wpisy

$$(20, 9), (19, 9), (18, 9), (17, 9), (16, 9), (15, 9), (14, 9), (13, 9), (9, 9).$$

Tylko ostatni z nich odpowiada ostatecznej najkrótszej odległości. Pozostałe elementy stają się wpisami nieaktualnymi, jednak nadal znajdują się w kopcu. Dlatego pobieranie nowych elementów z kopca staje się coraz bardziej kosztowne. Powoduje ono wiele operacji przesuwania elementów w celu przywrócenia własności kopca.

4.2.4 Odchylenie standardowe wersji z kopcem

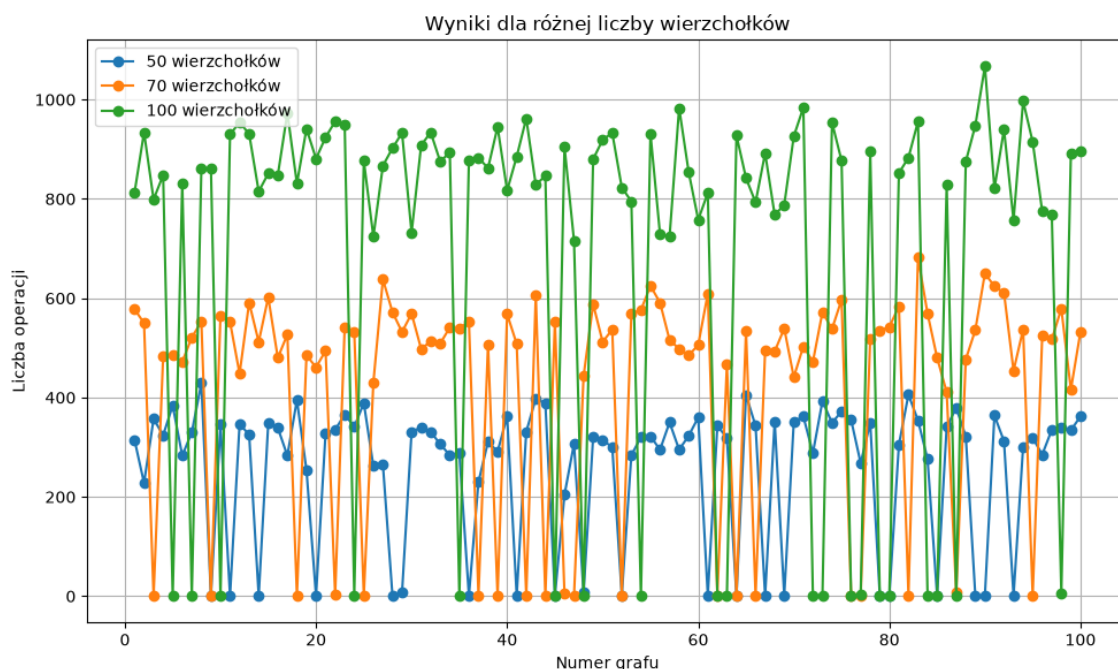


Rysunek 4.6: Odchylenie standardowe liczby operacji dla wersji wykorzystującej kopiec binarny.

Na rys. 4.6 widoczny jest wzrost odchylenia standardowego dla trzech losowo generowanych typów grafów. Jest to konsekwencją losowego doboru krawędzi, jak i ich wag.

W przypadku grafu średnio gęstego i gęstego różne wagi prowadzą do różnej liczby udanych relaksacji. Jeżeli pierwsza znaleziona droga do wierzchołka jest już stosunkowo krótka, liczba kolejnych aktualizacji wag będzie niewielka. Jeżeli natomiast początkowo

znaleziona zostanie droga o dużym koszcie, może być ona poprawiana wielokrotnie. Każda poprawa powoduje wstawienie nowego elementu do kopca i zwiększa całkowitą liczbę operacji. Dla grafu rzadkiego dodatkowym czynnikiem jest fakt że wygenerowany graf rzadki nie zawsze jest spójny.

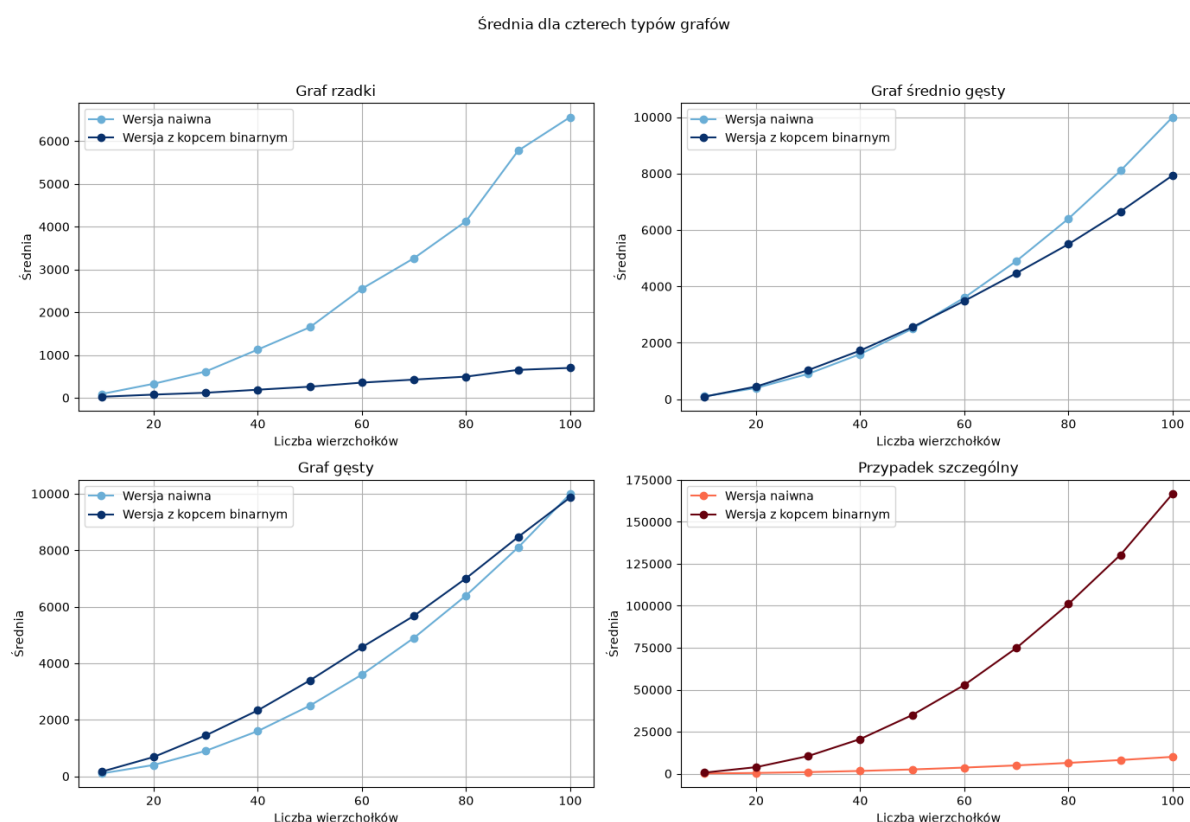


Rysunek 4.7: Liczba operacji wersji z kopcem binarnym dla kolejnych losowo wygenerowanych grafów rzadkich.

Jak pokazano na rys. 4.7, podobnie jak w przypadku wersji naiwnej, wyniki tworzą dwie wyraźne grupy. Dla 100 wierzchołków duża część wyników znajduje się w przedziale około 800–950 operacji, a część przyjmuje wartości bliskie zeru lub znacznie mniejsze od głównej grupy. Dla 70 wierzchołków dominują wartości około 500–600, a dla 50 wierzchołków około 300–350.

W przypadku szczególnym odchylenie standardowe wynosi dokładnie zero bo nie występuje tam losowanie struktury ani wag grafu. Dla ustalonego n każdy wygenerowany przypadek ma dokładnie ten sam układ krawędzi i wag, a algorytmu wykonuje tę samą liczbę operacji.

4.3 Bezpośrednie porównanie średnich wartości



Rysunek 4.8: Porównanie średniej liczby operacji wersji naiwnej i wersji z kopcem dla poszczególnych typów grafów.

Rysunek 4.8 pozwala bezpośrednio porównać obie implementacje dla każdego rodzaju grafu.

Dla grafu rzadkiego zastosowanie kopca przynosi największą korzyść. Niewielka liczba krawędzi ogranicza liczbę relaksacji, czyli też liczbę nowych wpisów umieszczanych w kopcu. Na tym samym grafie wersja naiwna przy każdej iteracji nadal przegląda całą tablicę wierzchołków. W przypadku grafu o średniej gęstości przewaga kopca jest mniejsza jednak powiększa się ona wraz ze wzrostem liczby krawędzi. Natomiast dla grafu gęstego wartości obu wariantów stają się do siebie zbliżone a dla 100 wierzchołków wersja naiwna zaczyna być lepsza od kopcowej. Wynika to z coraz większej liczby relaksacji przy zwiększającej się liczbie wierzchołków. Zupełnie inne wyniki widoczne są dla przypadku szczególnego. Wersja naiwna pozostaje na poziomie n^2 , a koszt wersji z kopcem bardzo szybko rośnie. Dla $n = 100$ wartości wynoszą odpowiednio około

10000

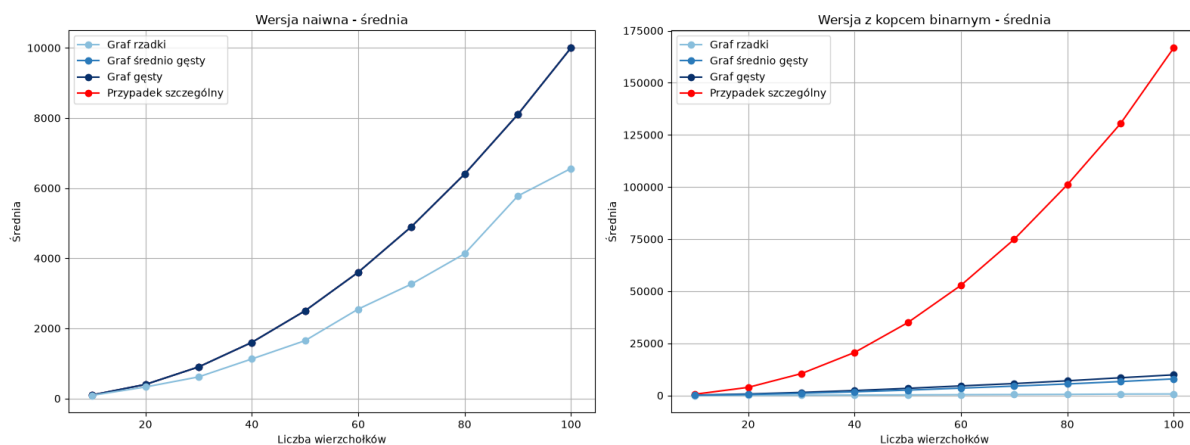
oraz

166000.

Stosunek tych wartości wynosi w przybliżeniu

$$\frac{166000}{10000} \approx 16.6. \quad (4.8)$$

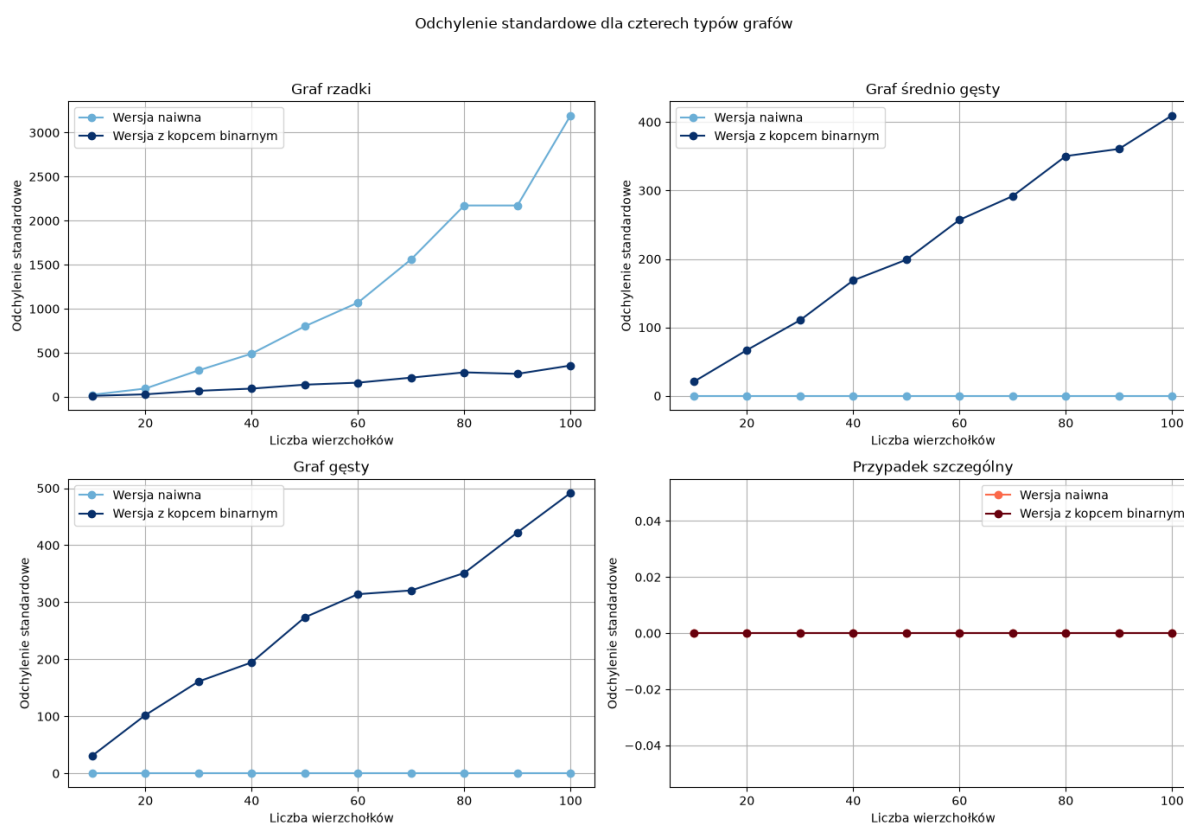
Oznacza to, że w algorytm wykorzystujący kopiec wykonuje w tym przypadku około 16.6 razy więcej operacji niż w wersji naiwnej.



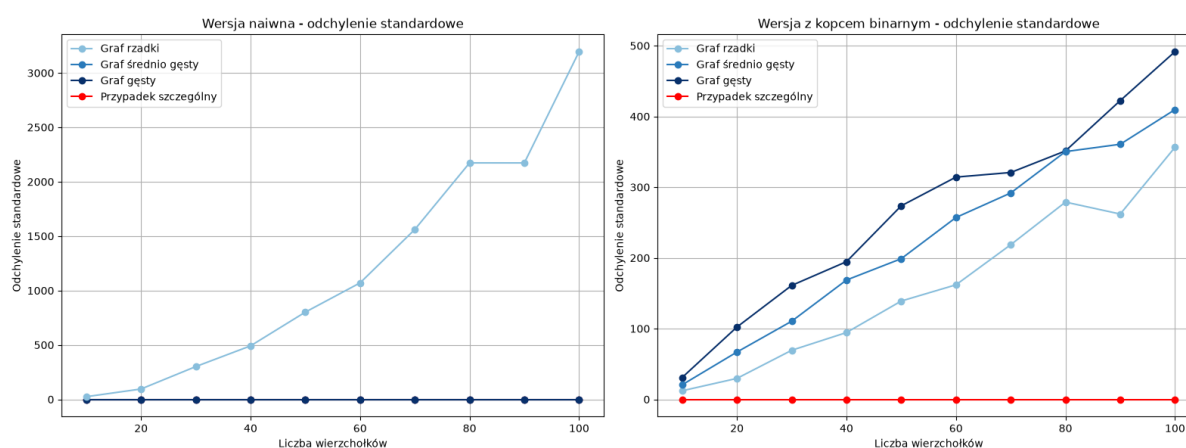
Rysunek 4.9: Porównanie średniej liczby operacji dla wszystkich typów grafów i obu implementacji.

Na rys. 4.9 jeszcze lepiej jest widoczna różnica pomiędzy przypadkiem szczególnym a pozostałymi grafami.

4.4 Porównanie odchylenia standardowego



Rysunek 4.10: Porównanie odchylenia standardowego obu implementacji dla poszczególnych typów grafów.



Rysunek 4.11: Porównanie odchylenia standardowego dla wszystkich typów grafów.

Rysunki 4.10 i 4.11 bardzo dobrze pokazują różnicę pomiędzy stabilnością obu metod zliczania.

W wersji naiwnej dla grafów spójnych liczba operacji zależy wyłącznie od liczby wierzchołków. Dla grafu średnio gęstego, gęstego i przypadku szczególnego odchylenie standardowe wynosi zero. Jedynie dla grafu rzadkiego odchylenie standardowe jest bardzo duże. Na podstawie rys. 4.2 można stwierdzić, że jest on spowodowany różnicami pomiędzy losowymi grafami. Czasami algorytm kończy działanie bardzo wcześnie bo nie może dotrzeć do wielu wierzchołków.

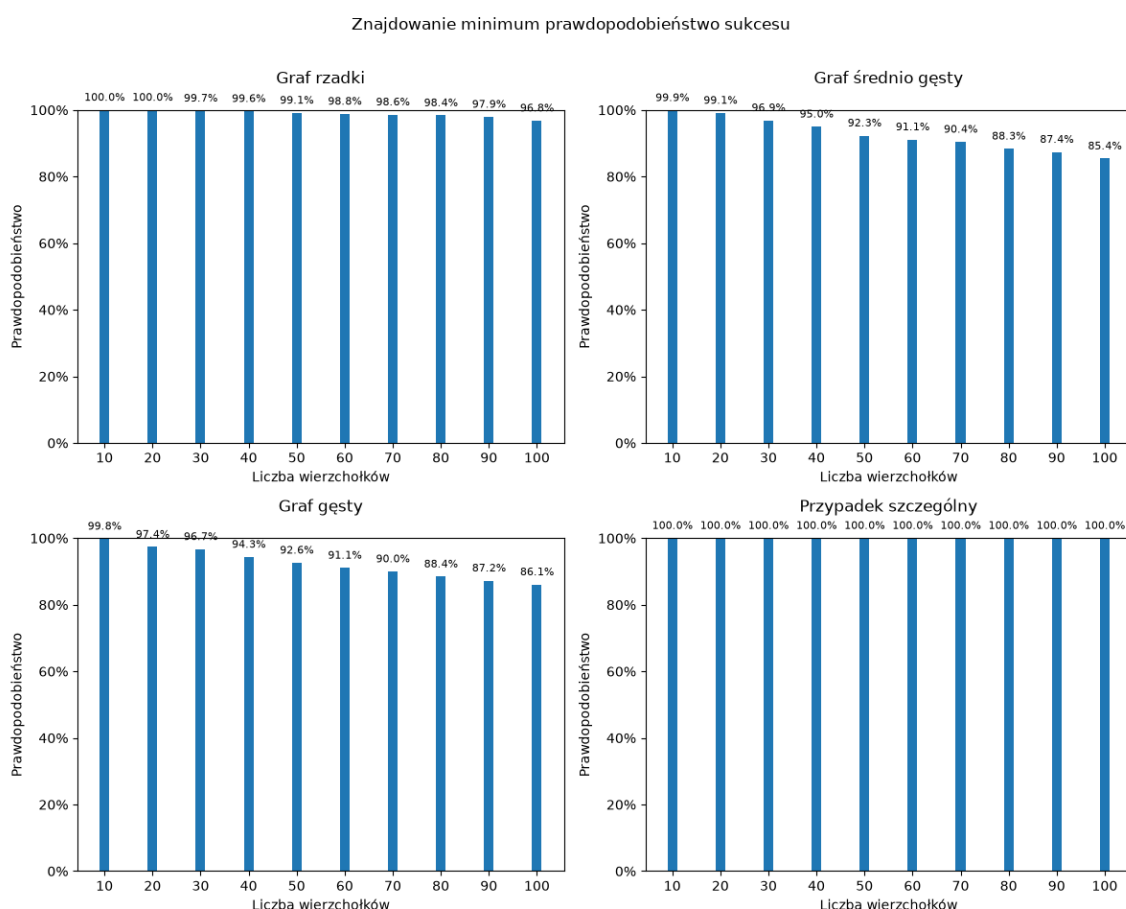
W wersji z kopcem wyniki zależą znacznie bardziej od konkretnego układu wag i liczby udanych relaksacji. Nawet dla grafów o tej samej liczbie wierzchołków i krawędzi różna kolejność poprawiania odległości może spowodować inną liczbę operacji wstawiania i pobierania wierzchołków z kopca. W przypadku grafu rzadkiego dodatkowo występuje ten sam efekt związany z brakiem spójności grafów co w wersji naiwnej. Powoduje to występowanie zarówno bardzo niskich i wysokich wyników, co jest widoczne na rys. 4.7.

Natomiast dla przypadku szczególnego zarówno w wersji z kopcem jak i naiwnej odchylenie standardowe jest równe zero. Jest to spowodowane specyficzną konstrukcją grafu która wymuszając maksymalną liczbę relaksacji poprzez odpowiedni dobór wag sprawia że algorytm działa zawsze w ten sam powtarzalny sposób.

Analiza poprawności wersji kwantowej

W niniejszym rozdziale przeprowadzono analizę poprawności kwantowej wersji algorytmu Dijkstry wykorzystującej algorytm wyszukiwania minimum Dürra-Høyer. Analizie poddano cztery rodzaje grafów: graf rzadki, graf o średniej gęstości, graf gęsty oraz przypadek szczególny. W pierwszej kolejności przeanalizowano skuteczność samego algorytmu wyszukiwania minimum, a następnie skuteczność całego algorytmu Dijkstry. Wyniki przedstawiono osobno dla wersji z limitem liczby operacji oraz bez limitu. W dalszej części rozdziału przeanalizowano zależność pomiędzy błędnym znalezieniem minimum a poprawnością końcowego wyniku algorytmu Dijkstry.

5.1 Skuteczność wyszukiwania minimum z limitem

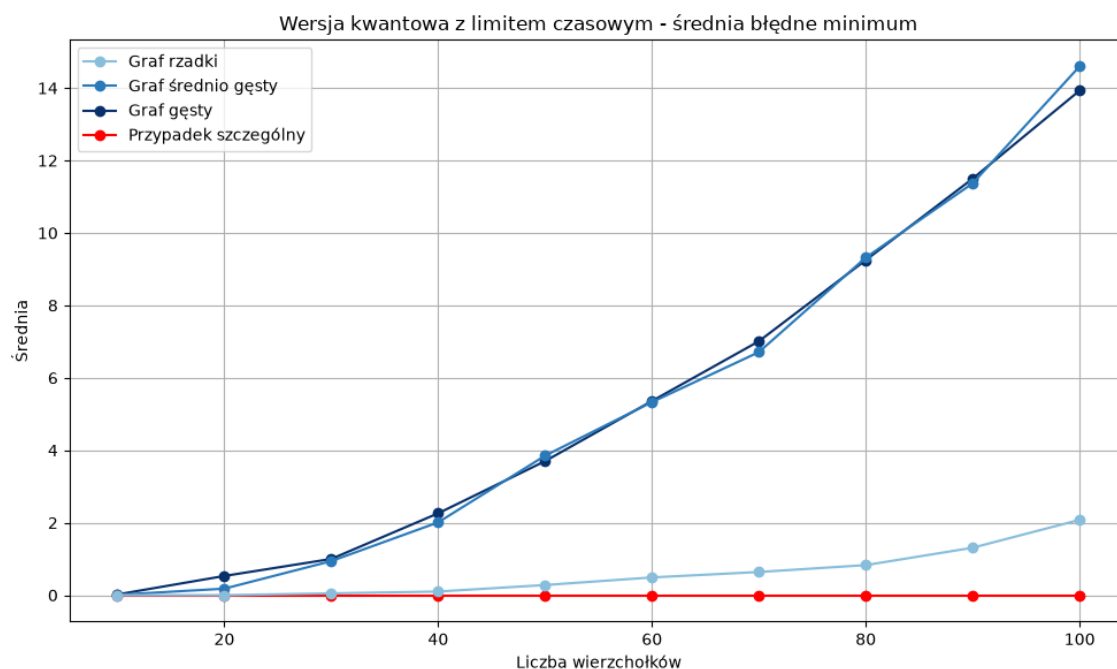


Rysunek 5.1: Prawdopodobieństwo poprawnego znalezienia minimum przez algorytm Dürre-Høyer z limitem liczby operacji.

Na rys. 5.1 przedstawiono prawdopodobieństwo poprawnego znalezienia minimum przy zastosowaniu limitu liczby operacji.

Najbardziej widoczny jest spadek prawdopodobieństwa poprawnego wyszukania minimum wraz ze wzrostem liczby wierzchołków. Zjawisko to występuje przede wszystkim dla grafów o średniej gęstości oraz grafów gęstych. Dla grafów o małej liczbie wierzchołków skuteczność pozostaje bliska 100%, a dla większych grafów stopniowo maleje. W przypadku grafu rzadkiego spadek prawdopodobieństwa poprawnego wyniku jest znacznie mniejszy. Skuteczność pozostaje bardzo wysoka nawet dla największych grafów. Natomiast przypadek szczególny zachowuje się inaczej od pozostałych typów grafów. Dla wszystkich analizowanych rozmiarów algorytm odnajduje poprawne minimum.

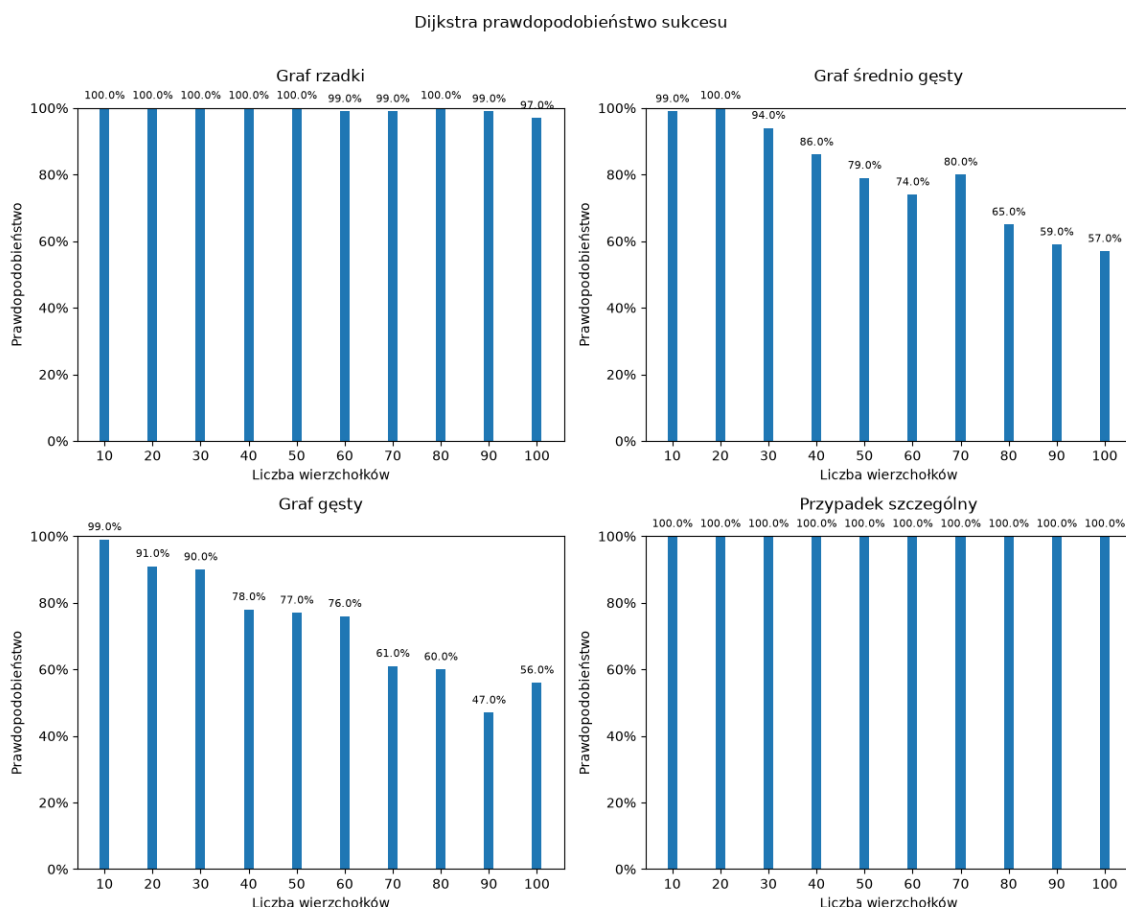
Uzyskane wyniki pokazują, że wpływ limitu nie zależy wyłącznie od liczby wierzchołków. Znaczenie ma też struktura grafu i odległości między wierzchołkami, które stanowią dane wejściowe dla algorytmu wyszukiwania minimum.



Rysunek 5.2: Średnia liczba błędnych wyszukiwań minimum w zależności od liczby wierzchołków dla konfiguracji z limitem.

Na rys. 5.2 przedstawiono średnią liczbę błędnych wyszukiwań minimum w zależności od liczby wierzchołków. Wraz ze wzrostem rozmiaru grafu liczba błędów zauważalnie rośnie przede wszystkim dla grafów o średniej gęstości oraz grafów gęstych. Dla grafu rzadkiego wartości pozostają niewielkie, natomiast w przypadku szczególnym błędne wyszukiwania minimum nie występują.

5.2 Skuteczność algorytmu Dijkstry z limitem



Rysunek 5.3: Prawdopodobieństwo otrzymania poprawnego wyniku kwantowej wersji algorytmu Dijkstry z limitem liczby operacji.

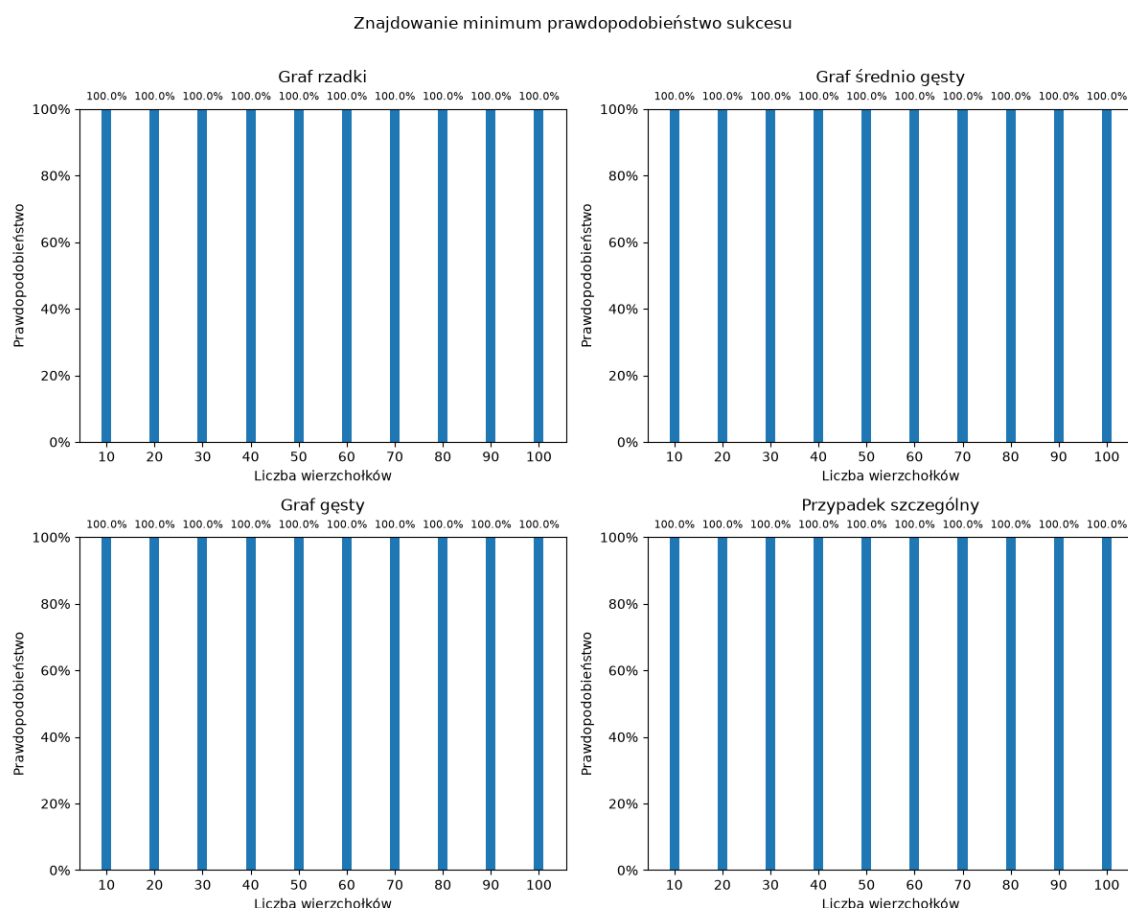
Na rys. 5.3 przedstawiono skuteczność całego algorytmu Dijkstry w wersji z limitem liczby operacji.

Widoczna jest podobna zależność jak w przypadku samego wyszukiwania minimum, ale spadek prawdopodobieństwa sukcesu jest znacznie większy. Dla grafu rzadkiego algorytm zachowuje wysoką poprawność ale dla grafów średnio gęstych oraz gęstych skuteczność maleje znacznie szybciej wraz ze wzrostem liczby wierzchołków.

Różnica ta wynika z wielokrotnego wykonywania wyszukiwania minimum podczas jednego uruchomienia algorytmu Dijkstry. Nawet jeżeli pojedyncze wyszukiwanie minimum ma wysokie prawdopodobieństwo sukcesu, każde kolejne wywołanie stanowi dodatkową możliwość wystąpienia błędu. Dlatego wraz ze wzrostem rozmiaru grafu coraz bardziej

widoczna staje się różnica pomiędzy skutecznością pojedynczej operacji wyszukiwania minimum a skutecznością całego algorytmu Dijkstry.

5.3 Skuteczność wyszukiwania minimum bez limitu

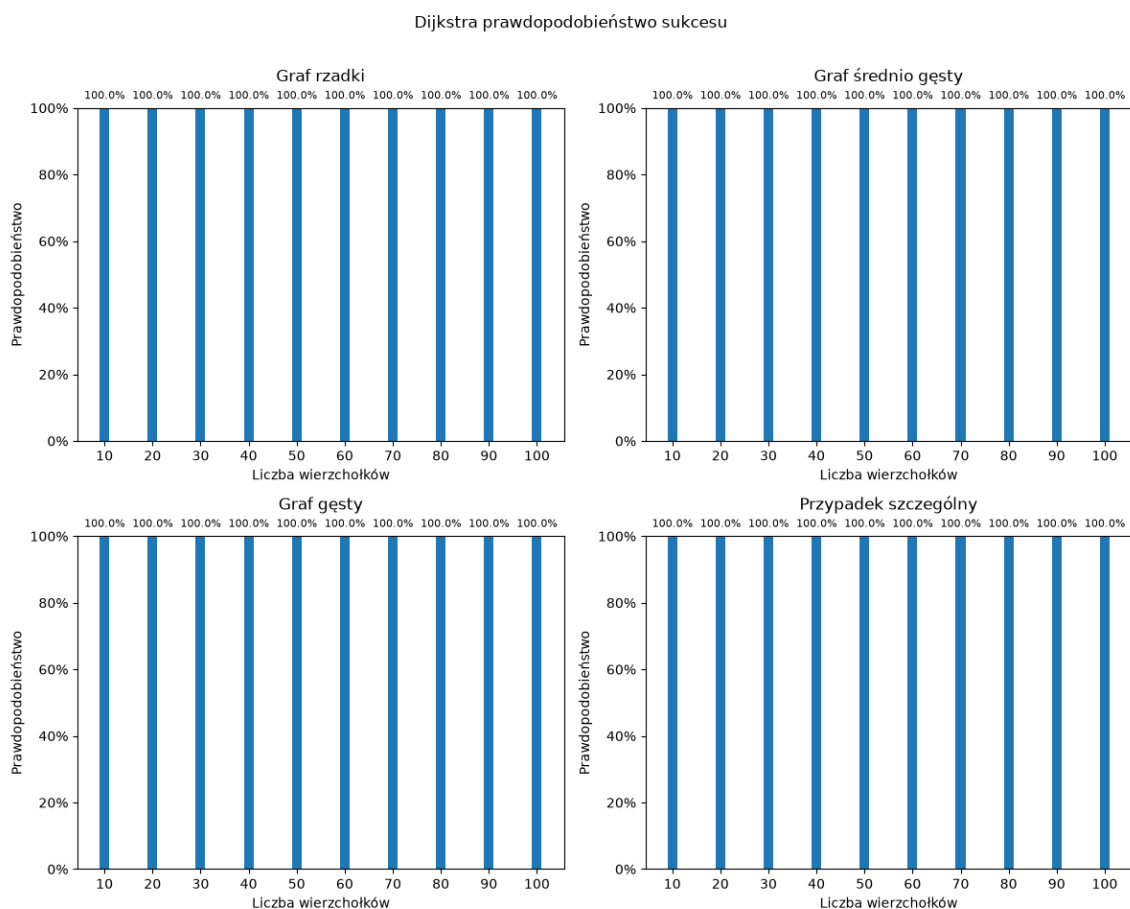


Rysunek 5.4: Prawdopodobieństwo poprawnego znalezienia minimum przez algorytm Dürra-Høyer bez limitu liczby operacji.

Na rys. 5.4 przedstawiono skuteczność wyszukiwania minimum bez ograniczenia liczby operacji.

W tej wersji dla wszystkich typów grafów oraz wszystkich badanych rozmiarów otrzymano pełną skuteczność. W porównaniu do wersji z limitem widać, że ograniczenie liczby wykonywanych operacji odpowiada za obserwowane wcześniej błędy. Gdy algorytm wyszukiwania minimum może wykonać nieograniczoną ilość prób, zwiększanie liczby wierzchołków nie prowadzi do pogorszenia poprawności wyszukiwania minimum.

5.4 Skuteczność algorytmu Dijkstry bez limitu

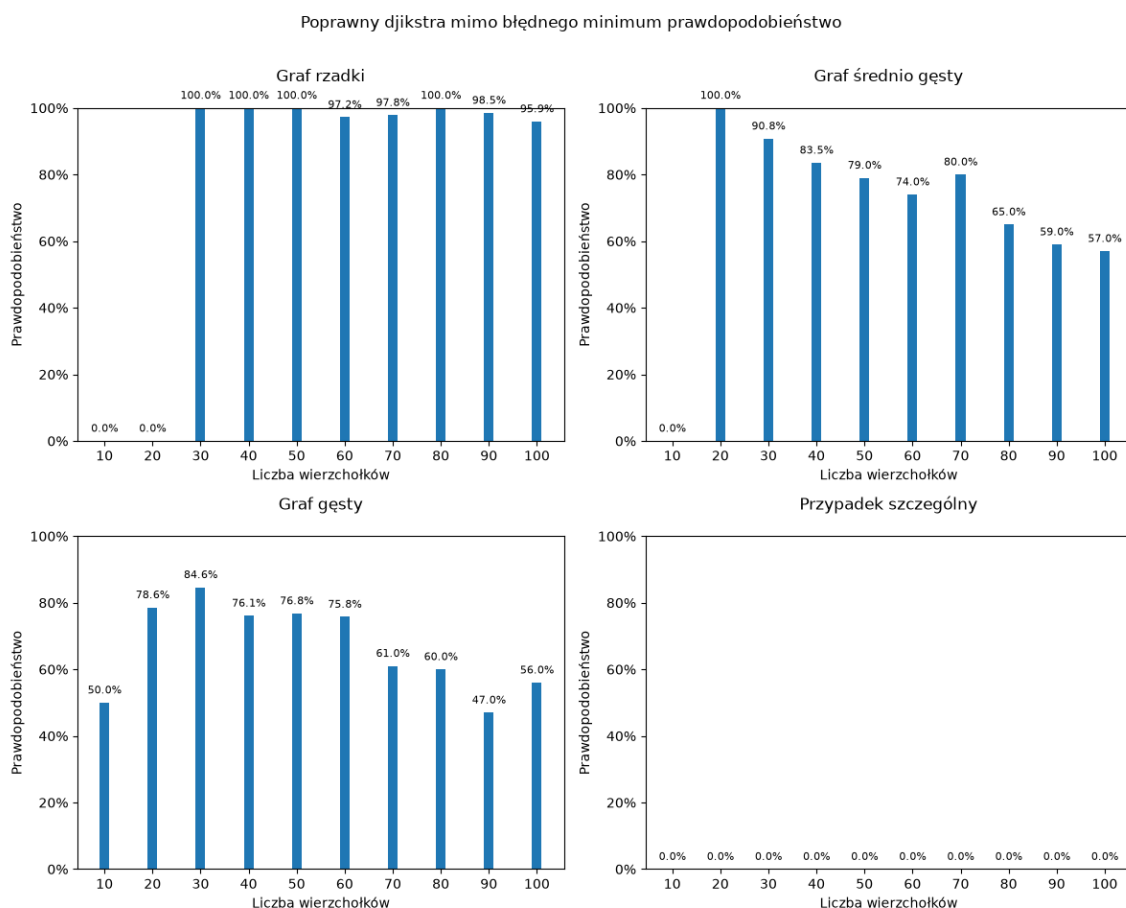


Rysunek 5.5: Prawdopodobieństwo otrzymania poprawnego wyniku kwantowej wersji algorytmu Dijkstry bez limitu liczby operacji.

Na rys. 5.5 przedstawiono skuteczność całego algorytmu Dijkstry bez ograniczenia liczby operacji wyszukiwania minimum.

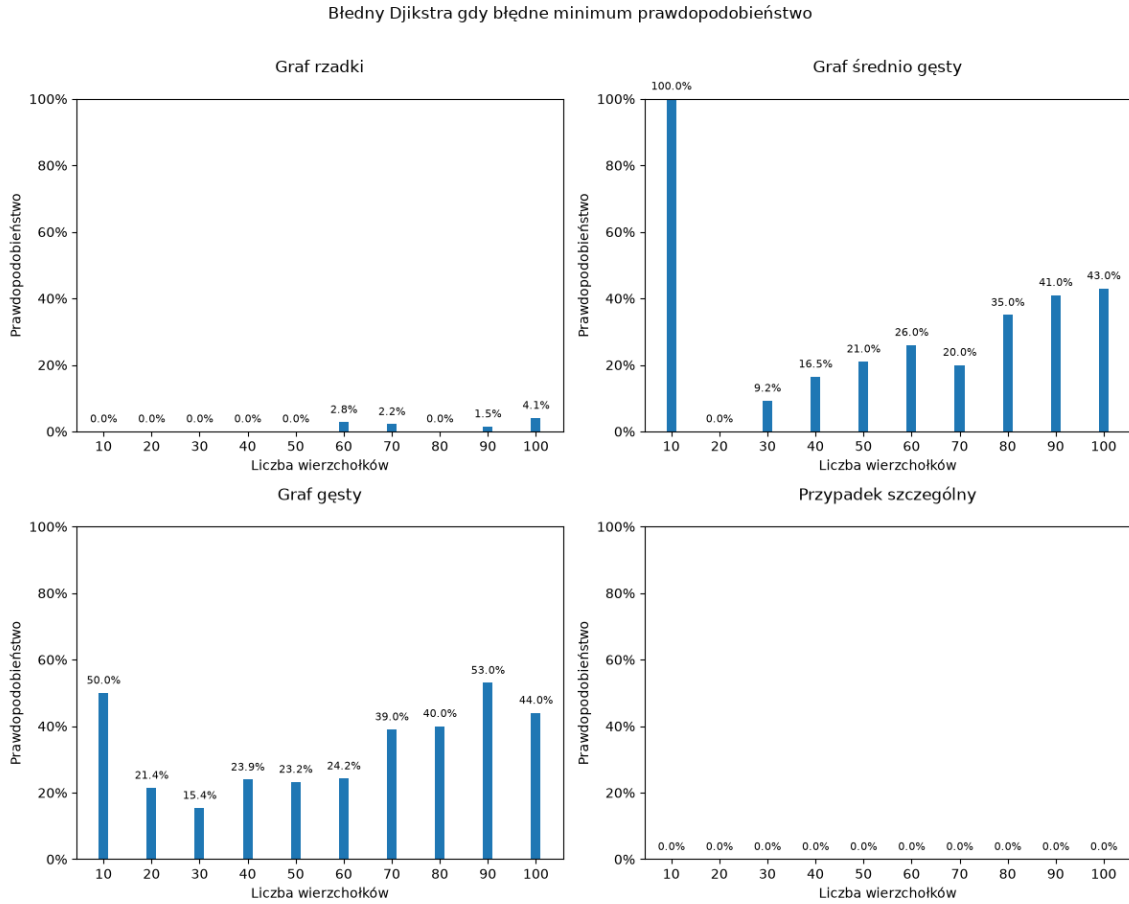
Podobnie jak w przypadku samego algorytmu Dürra-Høyer, dla wszystkich rodzajów grafów otrzymano pełną poprawność.

5.5 Zależność pomiędzy błędnym minimum a wynikiem algorytmu Dijkstry



Rysunek 5.6: Prawdopodobieństwo otrzymania poprawnego wyniku algorytmu Dijkstry pomimo wystąpienia błędnego minimum dla konfiguracji z limitem.

Na rys. 5.6 przedstawiono przypadki, w których mimo wystąpienia błędnego minimum końcowy wynik algorytmu Dijkstry pozostaje poprawny.



Rysunek 5.7: Prawdopodobieństwo otrzymania błędnego wyniku algorytmu Dijkstry w przypadku wystąpienia błędnego minimum dla konfiguracji z limitem.

Rys. 5.7 przedstawia tę samą zależność od strony błędnych wyników końcowych.

5.5.1 Dlaczego błędne minimum nie zawsze prowadzi do błędnego wyniku

W klasycznym algorytmie Dijkstry w każdej iteracji wybierany jest kolejny nieodwiedzony wierzchołek o najmniejszej odległości od aktualnego wierzchołka. Następnie wykonywana jest relaksacja jego krawędzi.

Aktualizacja odległości zachodzi tylko wtedy, gdy

$$d(v) > d(u) + w(u, v). \quad (5.1)$$

W wersji kwantowej jeżeli algorytm wyszukiwania minimum Dürra-Høyer'a wybierze wierzchołek o nie najmniejszej odległości, nie oznacza to jeszcze, że końcowy wynik bę-

dzie niepoprawny. Dzięki relaksacji optymalne ścieżki między wierzchołkami mogą być poprawione. Oprócz tego w grafie może też istnieć wiele ścieżek prowadzących do tego samego wierzchołka. Relaksacje mogą wtedy doprowadzić do poprawnych wartości, nawet jeśli wcześniej wykonano niepoprawny wybór.

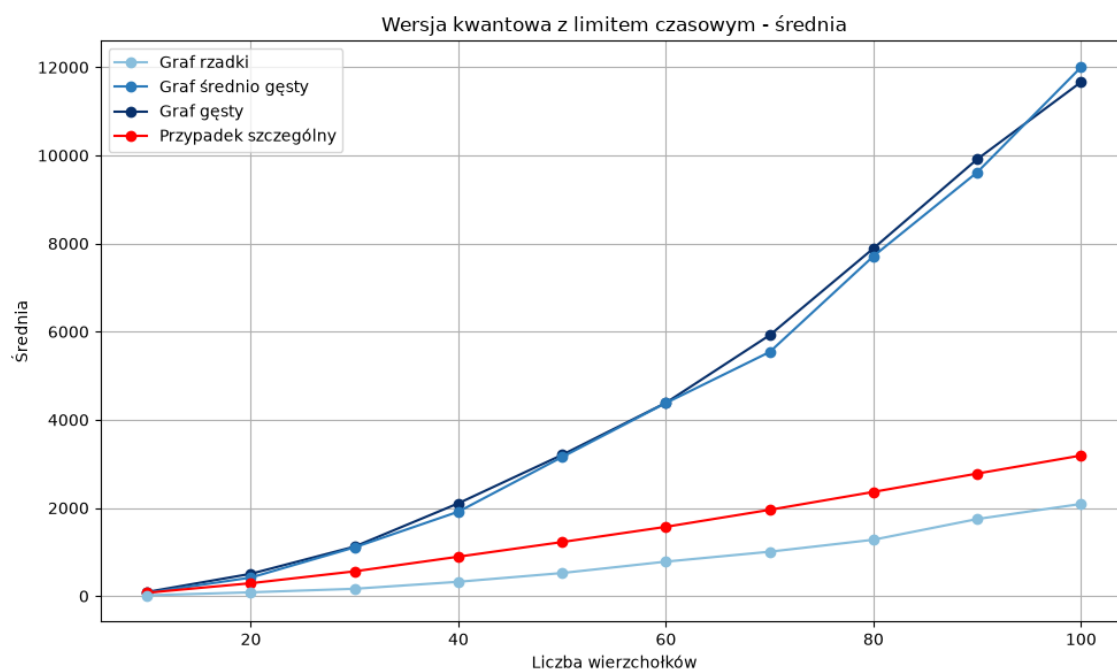
Analiza wyników implementacji kwantowej

W niniejszym rozdziale przeprowadzono analizę liczby operacji wykonywanych przez kwantową wersję algorytmu Dijkstry wykorzystującą algorytm wyszukiwania minimum Dürra-Høyer. Analizie poddano te same cztery rodzaje grafów co w pozostałej części pracy: graf rzadki, graf o średniej gęstości, graf gęsty oraz przypadek szczególny.

Liczba operacji wersji kwantowej została wyznaczona zgodnie ze sposobem zliczania opisanym w rozdziale dotyczącym implementacji. Uwzględnia ona koszt przygotowania rejestru w stanie superpozycji oraz liczbę wykonanych iteracji algorytmu Grovera.. Wyniki przedstawiono osobno dla wersji z limitem liczby operacji oraz bez limitu.

6.1 Wyniki dla wersji z limitem

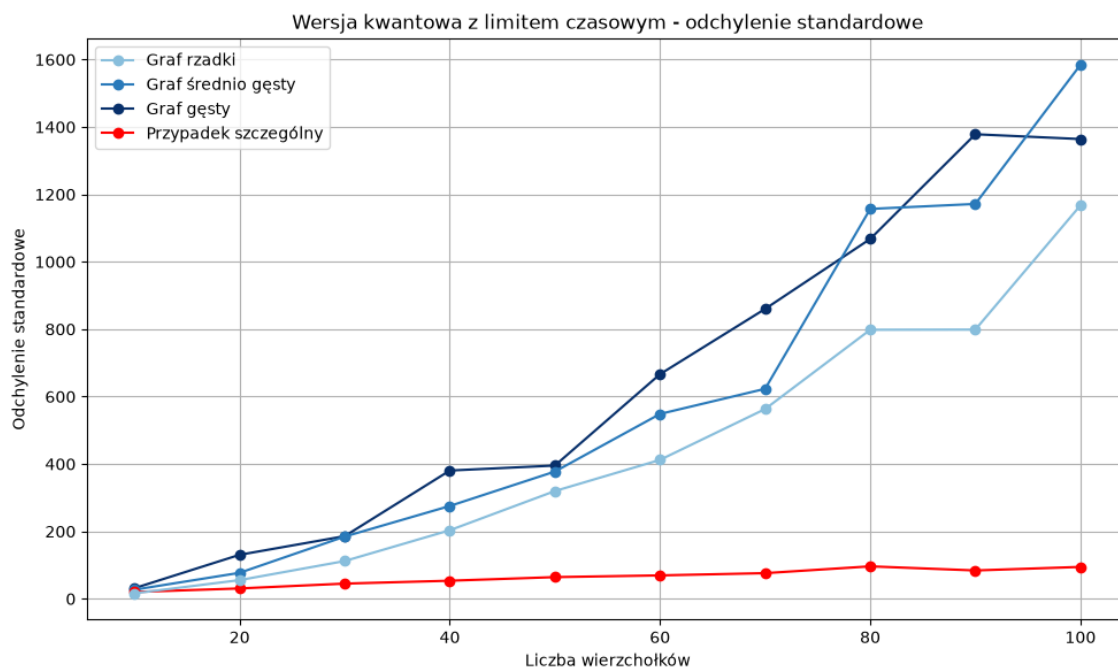
6.1.1 Średnia liczba operacji



Rysunek 6.1: Średnia liczba operacji kwantowej wersji algorytmu Dijkstry z limitem liczby operacji.

Na rys. 6.1 przedstawiono średnią liczbę operacji dla wersji z limitem. Dla wszystkich typów grafów liczba operacji rośnie wraz ze wzrostem liczby wierzchołków. Najmniejszy koszt występuje dla grafu rzadkiego, większe wartości obserwowane są dla przypadku szczególnego, natomiast największą liczbę operacji uzyskują grafy średnio gęste i gęste. Przypadek szczególny, mimo że ma tę samą liczbę krawędzi co graf gęsty, potrzebuje znacznie mniejszej liczby operacji niż grafy średnio gęste i gęste. Pokazuje to, że koszt kwantowego wyszukiwania minimum w algorytmie Dijkstry zależy nie tylko od liczby krawędzi, ale też od wag krawędzi.

6.1.2 Odchylenie standardowe

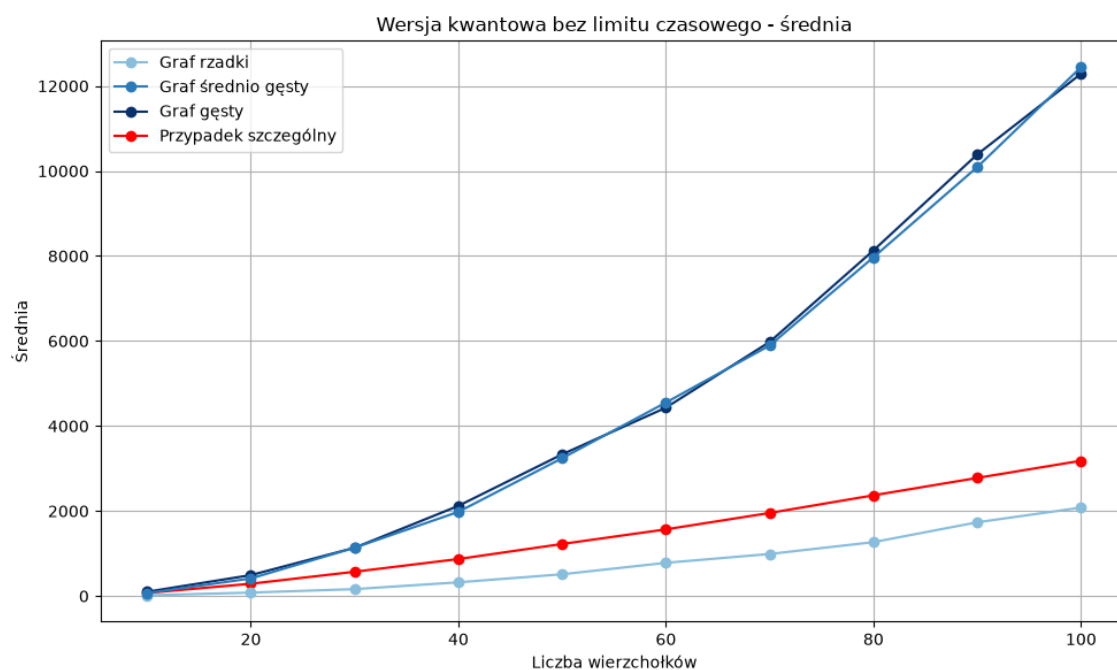


Rysunek 6.2: Odchylenie standardowe liczby operacji kwantowej wersji algorytmu Dijkstry z limitem liczby operacji.

Na rys. 6.2 przedstawiono odchylenie standardowe liczby operacji dla wersji z limitem. Wzrost odchylenia standardowego wraz z liczbą wierzchołków wynika z probabilistycznego charakteru algorytmu Dürra-Høyer, w szczególności z losowego wyboru początkowego kandydata na minimum i losowej liczby iteracji wykonywanych w kolejnych próbach BBHT.

6.2 Wyniki dla wersji bez limitu

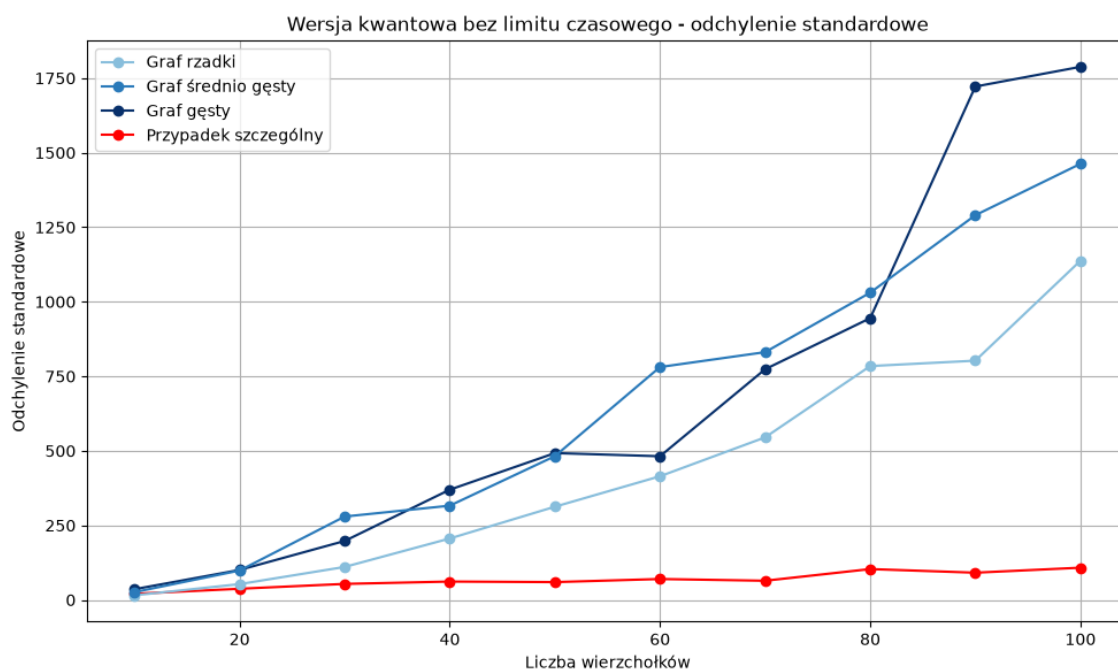
6.2.1 Średnia liczba operacji



Rysunek 6.3: Średnia liczba operacji kwantowej wersji algorytmu Dijkstry bez limitu liczby operacji.

Na rys. 6.3 przedstawiono średnią liczbę operacji dla konfiguracji bez limitu. Dla wszystkich typów grafów liczba operacji rośnie wraz ze wzrostem liczby wierzchołków, a zależności pomiędzy różnymi typami grafów są takie same jak dla wersji z limitem.

6.2.2 Odchylenie standardowe

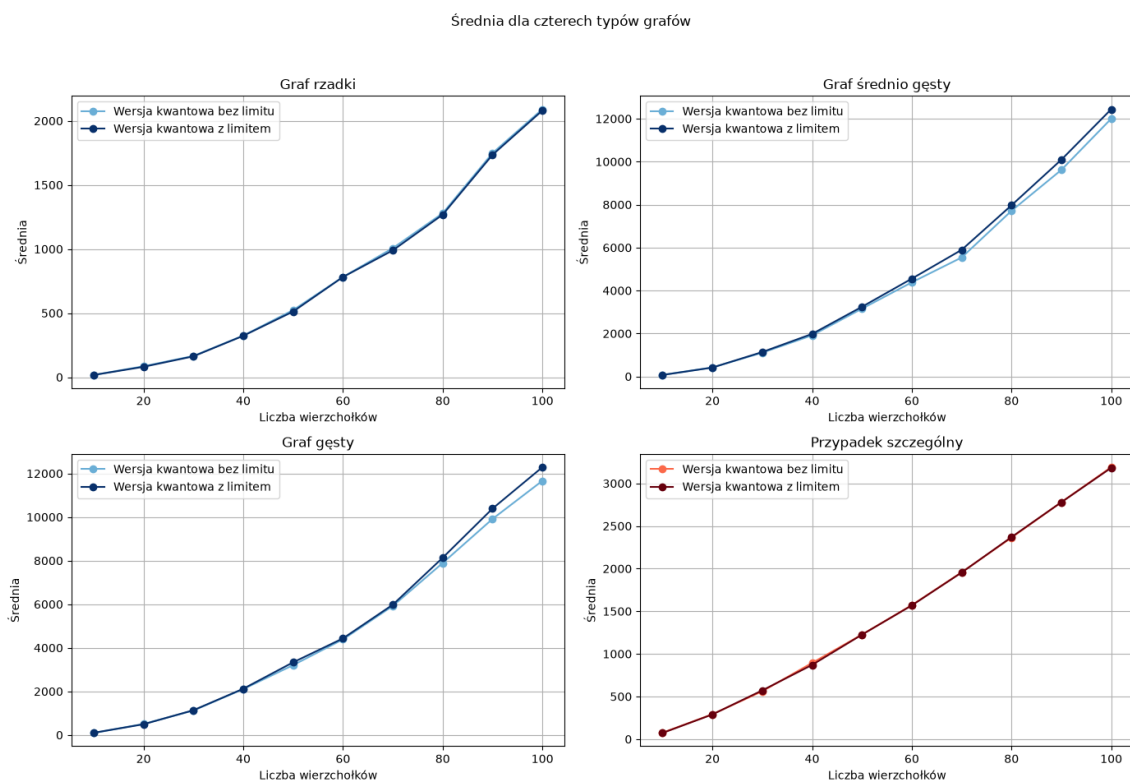


Rysunek 6.4: Odchylenie standardowe liczby operacji kwantowej wersji algorytmu Dijkstry bez limitu liczby operacji.

Na rys. 6.4 przedstawiono odchylenie standardowe liczby operacji dla konfiguracji bez limitu. Tutaj podobnie jak dla średniej ogólne tendencje są takie same jak dla wersji z limitem.

6.3 Porównanie wersji z limitem i bez limitu

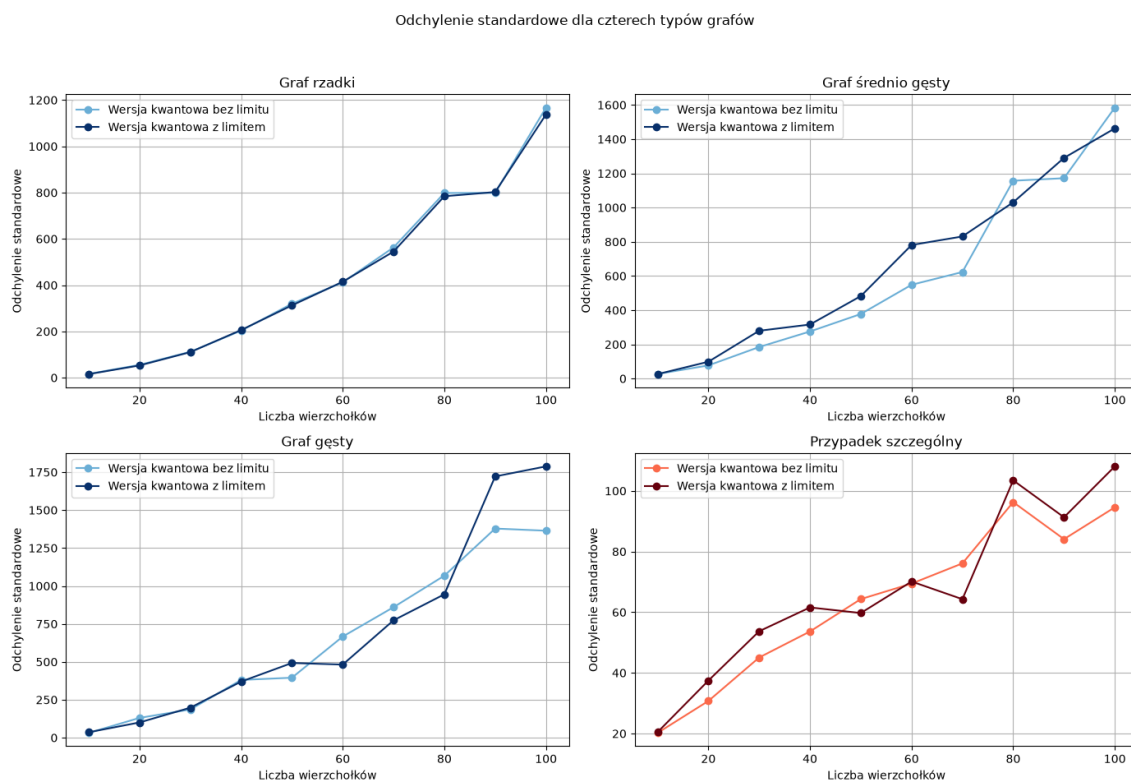
6.3.1 Porównanie średniej liczby operacji



Rysunek 6.5: Porównanie średniej liczby operacji wersji kwantowej z limitem i bez limitu dla czterech typów grafów.

Porównanie obu wersji przedstawiono na rys. 6.5. Dla grafu rzadkiego i przypadku szczególnego wykresy praktycznie się pokrywają dla wszystkich wierzchołków. Dla grafów średnio gęstych i gęstych różnice pomiędzy wersjami stają się bardziej widoczne wraz ze wzrostem liczby wierzchołków.

6.3.2 Porównanie odchylenia standardowego

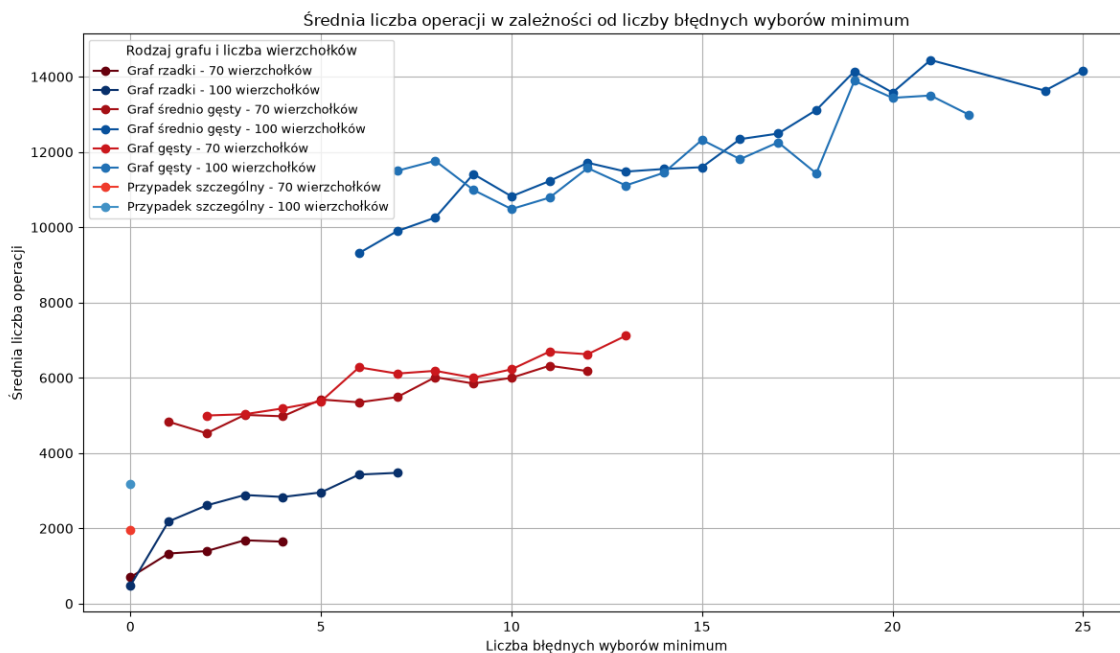


Rysunek 6.6: Porównanie odchylenia standardowego liczby operacji wersji kwantowej z limitem i bez limitu dla czterech typów grafów.

Rys. 6.6 umożliwia bezpośrednie porównanie odchylenia standardowego wyników dla obu wersji algorytmu Dijkstry. Dla grafu rzadkiego wartości są niemal identyczne. Większe różnice występują dla przypadku szczególnego oraz grafów średnio gęstych i gęstych.

6.3.3 Dlaczego koszt wersji z limitem i bez limitu jest podobny

Porównując wyniki liczby operacji dla wersji z limitem jak i bez limitu można zauważyć, że są one do siebie bardzo podobne, mimo że w analizie poprawności algorytmu obie wersje zachowują się inaczej. Wersja z limitem dla wielu przypadków zwraca niepoprawny wynik algorytmu Dijkstry, podczas gdy wersja bez limitu jest zawsze poprawna. Wyjaśnić to można, analizując liczbę operacji w zależności od liczby błędnych wyborów minimum.



Rysunek 6.7: Średnia liczba operacji w zależności od liczby błędnych wyborów minimum dla wybranych rodzajów i rozmiarów grafów.

Na rys. 6.7 przedstawiono zależność pomiędzy liczbą błędnych wyszukiwań minimum a średnią liczbą wykonanych operacji. Dla grafów rzadkich, średnio gęstych oraz gęstych można zaobserwować, że wraz ze wzrostem liczby błędnych wyszukiwań minimum rośnie również średni koszt operacji. Na tej podstawie można stwierdzić, że większej liczbie błędnych wyszukiwań minimum towarzyszy wyższy koszt. Wynika to z faktu, że błędne wyniki wyszukiwania mogą prowadzić do konieczności wykonania kolejnych wyszukiwań, zwiększając liczbę wykonywanych operacji.

Oprócz tego jak przedstawiono w rozdziale 5 stosunkowo niewielka część wyszukiwań minimum kończy się niepowodzeniem. Oznacza to, że zarówno w wersji z limitem, jak i bez limitu, głównym czynnikiem wpływającym na średnią liczbę operacji jest koszt poprawnie przeprowadzonych wyszukiwań. W przypadku wersji z limitem niewielka część błędnych wyszukiwań zostaje przerwana po przekroczeniu limitu, co powoduje wykonywanie kolejnych wyszukiwań i w konsekwencji zwiększa ich średni koszt. W wersji bez limitu takie wyszukiwania nie są przerywane, przez co mogą powodować większą liczbę operacji. W rezultacie średnie koszty uzyskane dla obu wersji są podobne.

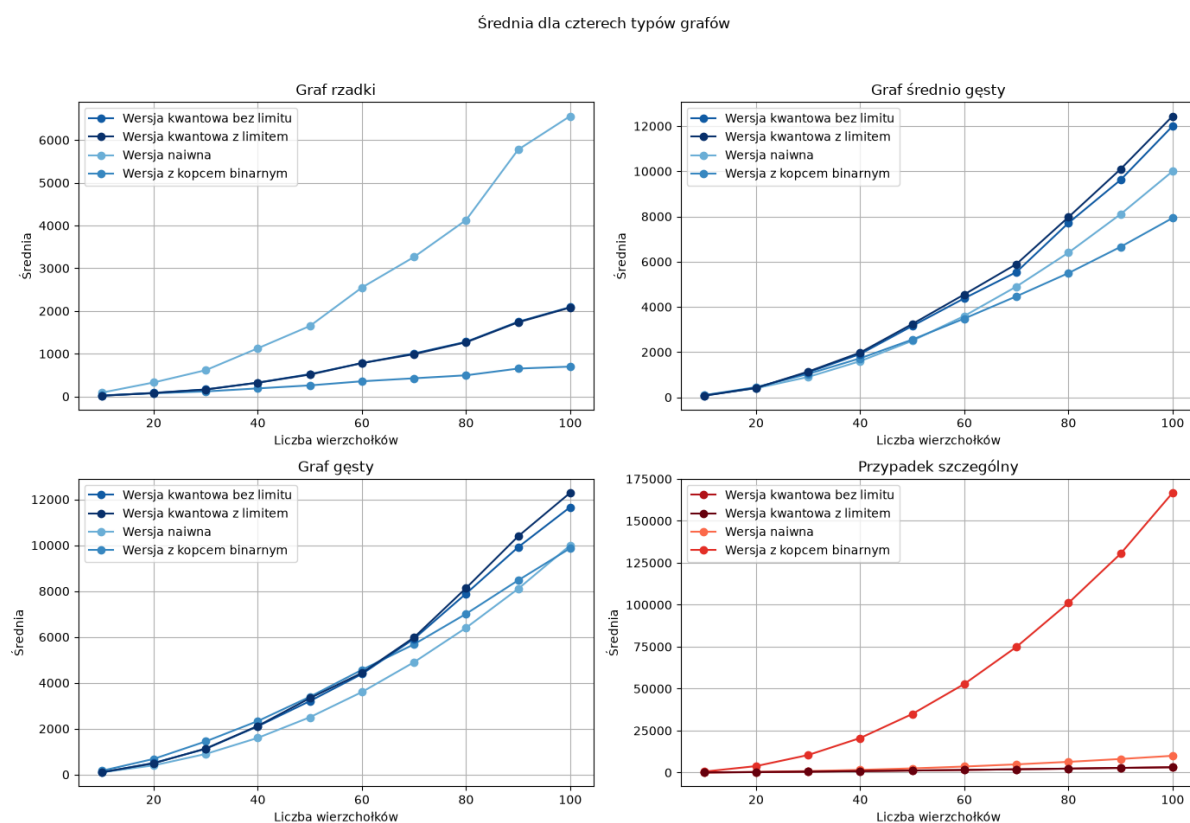
6.4 Zależność pomiędzy kosztem a poprawnością

Porównanie wyników z analizą poprawności pozwala zauważyć, że ogólnie wprowadzenie limitu nie wpływa pozytywnie na całość algorytmu. Jednak w szczegółach ten wniosek jest zależny od rodzaju rozpatrywanego grafu. Dla grafów rzadkich oraz przypadku szczególnego wprowadzenie limitu nie powoduje znaczących zmian, ponieważ zarówno koszt jak i poprawność jest podobna dla obu wersji. Natomiast dla grafów średnio gęstych i gęstych limit nie powoduje zmniejszenia wykonywanej przez algorytm liczby operacji a tylko znacząco zmniejsza jego skuteczność.

Porównanie implementacji klasycznych i kwantowej

W niniejszym rozdziale porównano wyniki wszystkich sprawdzanych implementacji algorytmu Dijkstry: wersji naiwnej, wersji wykorzystującej kopiec binarny oraz wersji kwantowej.

7.1 Porównanie średniej liczby operacji



Rysunek 7.1: Porównanie średniej liczby operacji wszystkich badanych implementacji dla czterech typów grafów.

Na rys. 7.1 przedstawiono porównanie średniej liczby operacji wszystkich badanych wersji algorytmu Dijkstry. Najważniejszą obserwacją jest brak jednej implementacji, która zachowywałaby się najlepiej dla każdego rodzaju grafu.

7.1.1 Graf rzadki

Dla grafów rzadkich średnia liczba operacji dla implementacji wykorzystującej kopiec binarny rośnie wyraźnie wolniej niż w przypadku pozostałych metod. Wynika to z niewielkiej liczby krawędzi, która ogranicza liczbę relaksacji oraz liczbę operacji wykonywanych w kopcu. Wersja naiwna zachowuje się gorzej, ponieważ podczas kolejnych wyborów minimum algorytm nadal sprawdza wszystkie wierzchołki. Wyniki wersji kwantowych znajdują się pomiędzy implementacją naiwną a kopcową.

7.1.2 Graf o średniej gęstości

Dla grafów o średniej gęstości różnice pomiędzy implementacjami są mniejsze niż dla grafów rzadkich. Jednak wraz ze wzrostem liczby wierzchołków różnice te się powiększają. Wersja z kopcem nadal lepiej wykorzystuje strukturę grafu niż wersja naiwna, ale jej przewaga jest mniejsza niż w przypadku grafów rzadkich ponieważ zwiększenie liczby krawędzi prowadzi do większej liczby relaksacji. Wersje kwantowe wykazują podobny do siebie przebieg a ich koszt rośnie szybciej niż koszt wersji z kopcem.

7.1.3 Graf gęsty

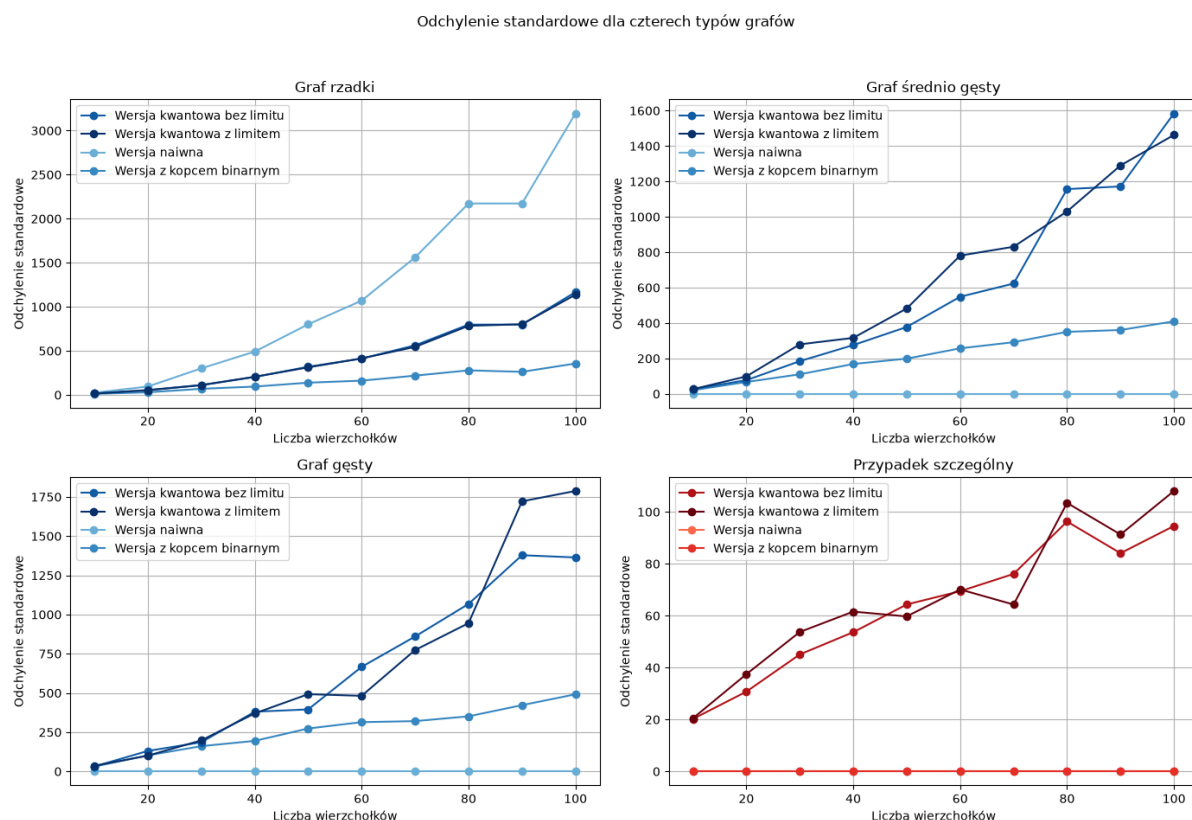
W przypadku grafów gęstych różnice pomiędzy metodami klasycznymi stają się mniejsze. Rosnąca liczba krawędzi powoduje zwiększenie liczby relaksacji, przez co kopiec stopniowo traci przewagę nad wersją naiwną widoczną dla grafów rzadkich. Wersje kwantowe również wykazują wyraźny wzrost średniej liczby operacji wraz z rozmiarem grafu. Dla większych grafów ich koszt zaczyna być wyższy niż dla wersji klasycznych.

7.1.4 Przypadek szczególny

Najbardziej wyróżniające się wyniki występują dla przypadku szczególnego. W wersja wykorzystującej kopiec binarny następuje bardzo szybki wzrost liczby operacji podczas

gdy średni koszt pozostałych wersji pozostaje niewielki. Przyczyną jest specjalna konstrukcja wag wymuszająca maksymalną liczbę relaksacji. Mechanizm ten został dokładnie opisany w rozdziale 4.2.3.

7.2 Porównanie odchylenia standardowego



Rysunek 7.2: Porównanie odchylenia standardowego liczby operacji wszystkich badanych implementacji dla czterech typów grafów.

Rysunek 7.2 pokazuje, że poszczególne implementacje różnią się nie tylko średnim kosztem, ale też odchyleniem standardowym określającym jak stabilne są wyniki.

7.2.1 Graf rzadki

Dla grafów rzadkich szczególnie duże odchylenie standardowe występuje w wersji naiwnej. Jest on spowodowany losową strukturą grafów oraz brakiem wymuszonej spójności. W zależności od wygenerowanego grafu algorytm może dotrzeć do dużej części wierzchołków albo zakończyć działanie znacznie wcześniej. To samo zjawisko zachodzi dla wersji

z kopcem jednak w znacznie mniejszym stopniu. W przypadku implementacji kwantowej do zmienności spowodowanej strukturą grafu dochodzi probabilistyczny charakter BBHT i Dürra-Høyer. Powoduje to wzrost odchylenia standardowego wraz z rozmiarem grafu.

7.2.2 Graf o średniej gęstości

Dla grafów o średniej gęstości wersja naiwna jest bardzo stabilna, ponieważ dla tego rodzaju grafu algorytm Dijkstry działa według tego samego schematu, nie ma problemu ze spójnością. Wersja z kopcem wykazuje większą zmienność, liczba operacji zależy od kolejności i liczby udanych relaksacji. Największy rozrzut średniej liczby operacji zauważalny jest dla implementacji kwantowych. Wynika to z losowej liczby prób BBHT i Dürra-Høyer. Wraz ze wzrostem liczby wierzchołków efekt ten staje się coraz bardziej widoczny.

7.2.3 Graf gęsty

Dla grafów gęstych utrzymuje się podobna zależność jak dla grafów o średniej gęstości. Wersja naiwna pozostaje bardzo stabilna, podczas gdy wyniki wersji z kopcem zależą od relaksacji. W implementacji kwantowej rozrzut tak jak dla grafów o średniej gęstości jest znacznie większy od implementacji klasycznych.

7.2.4 Przypadek szczególny

Przypadek szczególny pozwala dobrze rozdzielić wpływ struktury grafu na rozrzut wyników. Ze względu na specyficzny typ grafu odchylenie standardowe dla wersji klasycznych jest równe zero niezależnie od liczby wierzchołków. Natomiast wersja kwantowa zachowuje niewielkie, lecz niezerowe odchylenie standardowe. Wynika ono z samego sposobu działania probabilistycznego wyszukiwania minimum.

Podsumowanie

Celem pracy było zaimplementowanie i porównanie różnych wersji wyszukiwania minimum w algorytmie Dijkstry. Analizie poddano klasyczną wersję naiwną, implementację wykorzystującą kopiec binarny oraz implementację kwantową opartą na algorytmie Dürra-Høyer, wykorzystującym algorytmy BBHT i Grovera. Analizę przeprowadzono dla kilku typów grafów różniących się gęstością, strukturą połączeń oraz sposobem doboru wag.

Przeprowadzone badania pokazują, że wybór najlepszego sposobu wyszukiwania minimum w algorytmie Dijkstry powinien być uzależniony od struktury grafu. Kopiec binarny bardzo dobrze wykorzystuje rzadkość grafu, ale traci swoją przewagę wraz z wzrastającą liczbą wierzchołków.. Wersja naiwna jest bardziej przewidywalna, ale nie korzysta bezpośrednio z małej liczby krawędzi. Implementacja kwantowa charakteryzuje się probabilistycznym kosztem i dla badanych grafów nie wykazuje przewagi na implementacjami klasycznymi.

Najważniejszym wnioskiem jest więc brak jednej uniwersalnie najlepszej implementacji. Otrzymane wyniki pokazują, że gęstość grafu, jego spójność, struktura wag oraz przebieg relaksacji mają duży wpływ na koszt wyszukiwania minimum. Analiza średniej liczby operacji, odchylenia standardowego i poprawności pozwala pełniej ocenić zalety oraz wady każdej z badanej wersji.

Opis metod i technologii wykorzystanych w pracy

9.1 Język programowania

W niniejszej pracy wykorzystany został język programowania Python w wersji 3.14.6 [10] jako główne narzędzie do implementacji opracowanego rozwiązania. Wybór ten wynikał z faktu, że jest to stabilna i dobrze przetestowana dystrybucja tego oprogramowania. Pozwala ona na korzystanie z wielu zewnętrznych bibliotek i paczek bez konfliktów związanych z niezgodnością wersji. Oprócz tego z każdą kolejną aktualizacją języka dodawane są do niego kolejne usprawnienia dlatego w miarę możliwości powinno używać się go w jak najnowszej wersji.

9.2 Zewnętrzne paczki pythonowe

W projekcie wykorzystano zestaw zewnętrznych bibliotek języka Python. Ich konkretne wersje zostały określone w pliku **requirements.txt**, dzięki czemu możliwe jest odtworzenie środowiska wykorzystanego w pracy.

- **qiskit 2.5.0** [11]
- **qiskit_algorithms 0.4.0** [12]
- **networkx 3.6.1** [13]
- **numpy 2.5.1** [14]
- **pandas 3.0.3** [15]
- **matplotlib 3.11.1** [16]

Instrukcja instalacji i obsługi

10.1 Instalacja

Repozytorium zawierające kod źródłowy znajduje się pod tym linkiem [17]. W celu uruchomienia aplikacji wymagane jest wcześniejsze zainstalowanie języka Python [10], a także dowolnego środowiska umożliwiającego edycję oraz wykonywanie programów napisanych w tym języku, takiego jak na przykład PyCharm [18] lub Visual Studio Code [19].

Następnie, w celu pobrania repozytorium na środowisko lokalne, należy otworzyć folder docelowy w terminalu i użyć następującej komendy:

```
git clone https://github.com/Sandpitturtleee/Thesis.git
```

Oprócz tego konieczne jest również zainstalowanie wszystkich wymaganych zewnętrznych paczek. Lista wykorzystywanych bibliotek wraz z ich wersjami znajduje się w pliku **requirements.txt**. Ich instalację można przeprowadzić za pomocą następującej komendy:

```
pip install -r requirements.txt
```

10.2 Instrukcja obsługi

Projekt został podzielony na trzy główne części odpowiadające kolejnym jego etapom. Jest to generowanie grafów, wykonywanie algorytmów Dijkstry oraz analiza otrzymanych

wyników. Parametry wykorzystywane podczas działania programu znajdują się w pliku **config.py**.

10.2.1 Generowanie grafów

Pierwszym etapem działania aplikacji jest przygotowanie grafów wykorzystywanych następnie podczas testowania algorytmów. Kod odpowiedzialny za ten proces znajduje się w katalogu **graphs_creation/src**. Głównym plikiem służącym do uruchomienia procesu generowania grafów jest **graphs_creation/src/main.py**. Program generuje grafy o parametrach określonych w konfiguracji projektu. W pliku **config.py** możliwe jest określenie maksymalnego rozmiaru grafu oraz liczby generowanych przypadków.

10.2.2 Analiza grafów

Kolejnym etapem jest wykonanie algorytmów wyznaczania najkrótszych ścieżek Dijkstry dla wcześniej przygotowanych grafów. Kod odpowiedzialny za ten proces znajduje się w katalogu **graphs_analysis/src**. Główny plik uruchamiający analizę znajduje się pod ścieżką **graphs_analysis/src/main.py**. W katalogu **graphs_analysis/src** implementacje zostały rozdzielone na dwie części: **standard** oraz **quantum**. Pierwsza z nich zawiera klasyczne implementacje algorytmu Dijkstry, natomiast druga odpowiada za implementację kwantową.

10.2.3 Analiza statystyczna wyników

Po przeprowadzeniu eksperymentów otrzymane wyniki mogą zostać poddane dalszej analizie. Do jej przeprowadzenia wykorzystywany jest plik **data_analysis/src/main_analysis.py** znajdujący się w katalogu **data_analysis/src**.

10.2.4 Generowanie wykresów

W celu graficznego przedstawienia rezultatów przeprowadzonych eksperymentów w projekcie przygotowany został osobny moduł odpowiedzialny za tworzenie wykresów. Głównym plikiem wykorzystywanym w tym celu jest **data_analysis/src/main_plots.py**. Po wykonaniu generowane są wykresy pozwalające na porównanie rezultatów otrzymanych dla różnych implementacji algorytmu oraz różnych rodzajów i rozmiarów grafów.

Bibliografia

- [1] Reinhard Diestel. *Graph Theory*. Springer-Verlag Berlin and Heidelberg GmbH & Co. KG, 5 edition, 2017. https://daiwz.net/course/disc_math/2023/Diestel_Graph_Theory.pdf.
- [2] Dijkstra, E.W. A note on two problems in connexion with graphs. *Numer. Math.* 1, 269–271, 1959. <https://ir.cwi.nl/pub/9256/9256D.pdf>.
- [3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009. <https://www.cs.mcgill.ca/~akroitt/math/compsci/Cormen%20Introduction%20to%20Algorithms.pdf>.
- [4] Christoph Dürr and Peter Høyer. A quantum algorithm for finding the minimum. <https://arxiv.org/abs/quant-ph/9607014>, 1996.
- [5] Lov K. Grover. A fast quantum mechanical algorithm for database search. *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, 1996. <https://arxiv.org/abs/quant-ph/9605043>.
- [6] P. Høyer M. Boyer, G. Brassard and A. Tapp. Tight bounds on quantum searching. *Fortschritte Der Physik*, 1996. <https://arxiv.org/abs/quant-ph/9605034>.
- [7] Sawerwain Marek Wiśniewska Joanna. *Informatyka kwantowa. Wybrane obwody i algorytmy*. Wydawnictwo Naukowe PWN, 1 edition, 2015.
- [8] B. Bollobas, O. Riordan. Metrics for sparse graphs. *Surveys in Combinatorics* 2009, LMS Lecture Notes Series 365, CUP 2009, pp. 211–287, 2008. <https://arxiv.org/abs/0708.1919>.
- [9] Wolfram MathWorld. Complete graph. <https://mathworld.wolfram.com/CompleteGraph.html>. Accessed: 04.08.2026.

- [10] Python. <https://www.python.org/downloads/release/python-3146/>.
- [11] Qiskit. <https://pypi.org/project/qiskit/2.5.0/>.
- [12] Qiskit algorithms. <https://pypi.org/project/qiskit-algorithms/>.
- [13] Networkx. <https://pypi.org/project/networkx/>.
- [14] Numpy. <https://pypi.org/project/numpy/2.5.1/>.
- [15] Pandas. <https://pypi.org/project/pandas/3.0.3/>.
- [16] Matplotlib. <https://pypi.org/project/matplotlib/>.
- [17] Repozytorium. <https://github.com/Sandpitturtleeee/Thesis>.
- [18] PyCharm. <https://www.jetbrains.com/pycharm/>.
- [19] Visual Studio Code. <https://code.visualstudio.com>.